



中山大學

SUN YAT-SEN UNIVERSITY

第六章 组合逻辑电路函数

于帅

计算机学院

2026年4月15日 Wednesday

主要内容

1 加法器

2 比较器

3 译码器

4 编码器

5 多路复用/分配器

主要内容

1 加法器

2 比较器

3 译码器

4 编码器

5 多路复用/分配器



加法器

- **加法器是构成算术运算器的基本单元**

- 两个二进制数之间的算术运算（加、减、乘、除），目前在数字计算机中都是化作若干步加法运算进行的。
- 1. 半加器
- 2. 全加器
- 3. 多位加法器（并行二进制加法器）
- 4. 异步进位与超前进位加法器



加法器

• 1. 半加器

- 如果不考虑来自低位的进位，将两个1位二进制数相加，称为半加。
- 实现半加运算的电路叫做**半加器**。

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 0$$

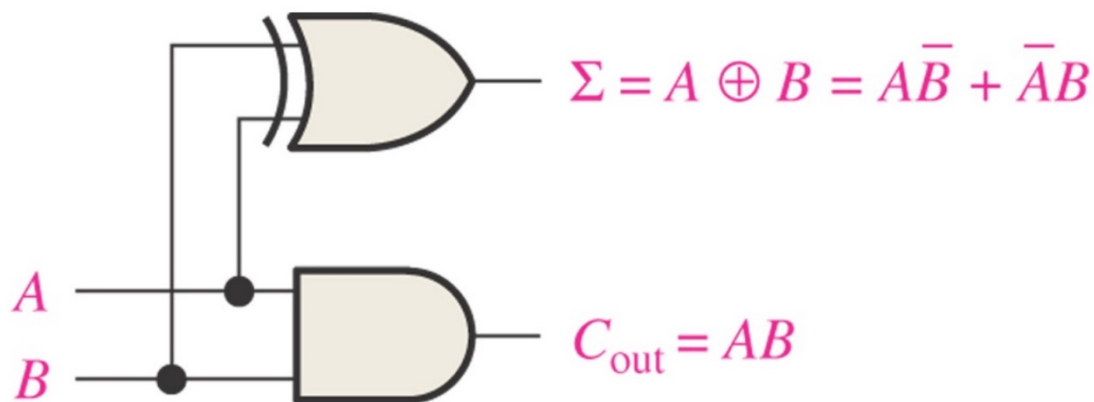
半加运算

半加器的逻辑符号



加法器

• 1. 半加器



A: 加数

B: 被加数

S: 和

C_{out}: 向高位的进位

半加器的逻辑框图

输 入		输 出	
A	B	S	CO
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

半加器的真值表



加法器

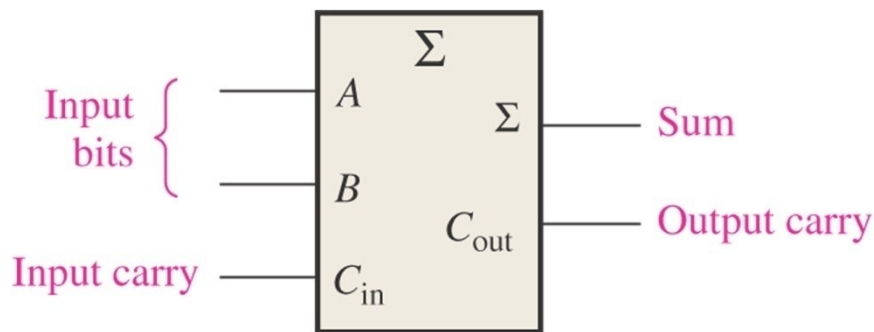
• 2. 全加器

- 全加运算：在两个多位二进制数相加时，除了最低位以外，每一位都应该考虑来自低位的进位，即将两个对应的加数和来自低位的进位3个数相加。

- 实现全加运算的电路叫做**全加器**。

1 1 1 1 0 0 0	
00001000	$(+8_{10})$
+11111101	(-3_{10}补码)
100000101	$(+5_{10})$

考虑来自低位进位的全加运算



全加器的逻辑符号



加法器

• 2. 全加器

输 入			输 出	
C_{in}	A	B	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

全加器的真值表

C_{in}	AB			
	00	01	11	10
0	0	1	0	1
1	1	0	1	0

S

C_{in}	AB			
	00	01	11	10
0	0	0	1	0
1	0	1	1	1

C_{out}

$$\Sigma = \overline{A}\overline{B}C_{in} + \overline{A}B\overline{C}_{in} + A\overline{B}\overline{C}_{in} + ABC_{in}$$

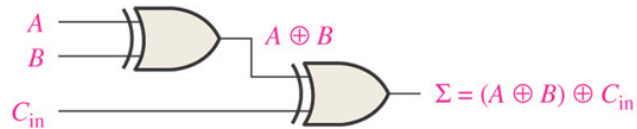
$$= (A \oplus B) \oplus C_{in}$$

$$C_{out} = \overline{A}BC_{in} + A\overline{B}C_{in} + AB\overline{C}_{in} + ABC_{in}$$

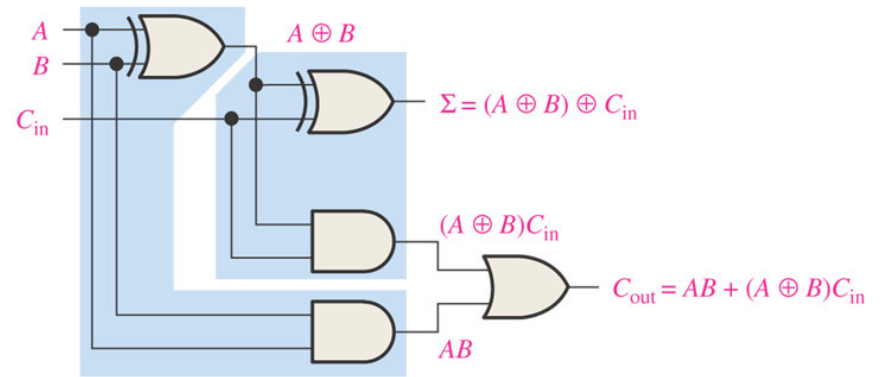
$$= AB + (A \oplus B)C_{in}$$



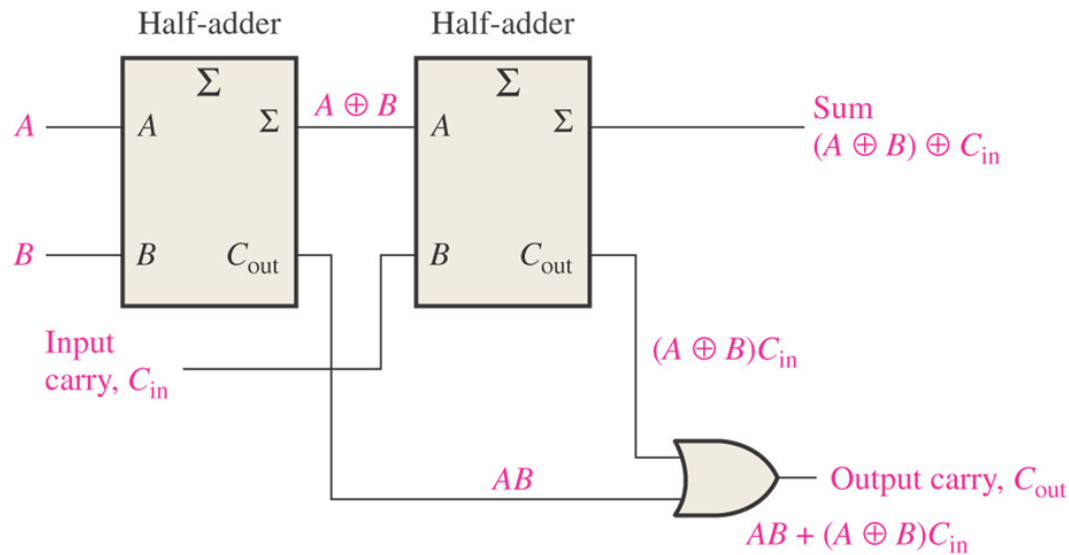
加法器



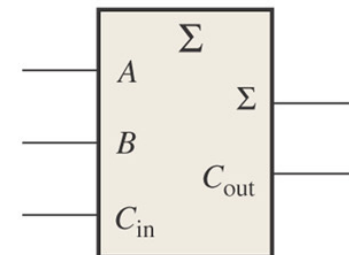
(a) Logic required to form the sum of three bits



(b) Complete logic circuit for a full-adder (each half-adder is enclosed by a shaded area)



(a) Arrangement of two half-adders to form a full-adder



(b) Full-adder logic symbol 9



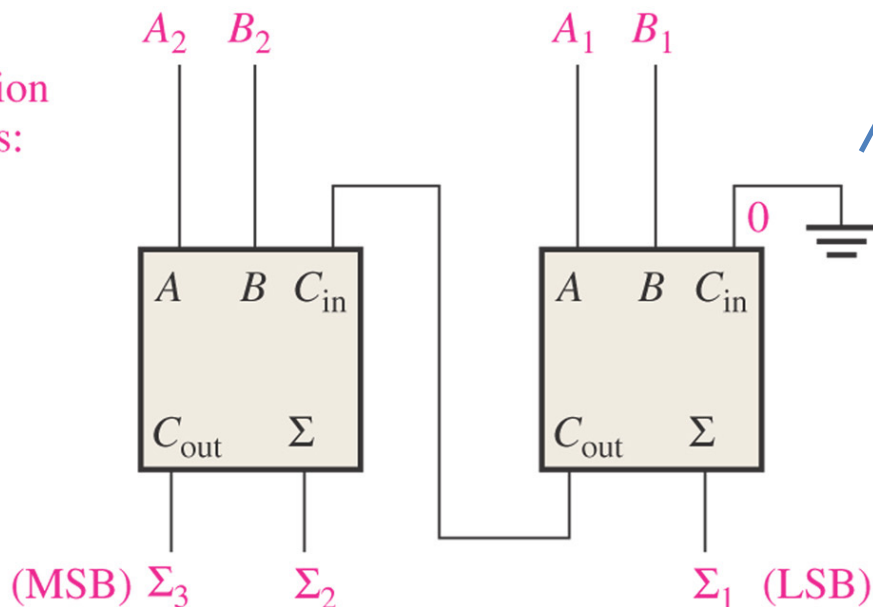
加法器

• 3. 多位加法器（并行二进制加法器）

- 依次将低位全加器的进位输出 C_{out} ，接到高位全加器的进位输入端 C_{in} ，即构成多位加法器。

General format, addition of two 2-bit numbers:

$$\begin{array}{r} A_2A_1 \\ + B_2B_1 \\ \hline \Sigma_3\Sigma_2\Sigma_1 \end{array}$$



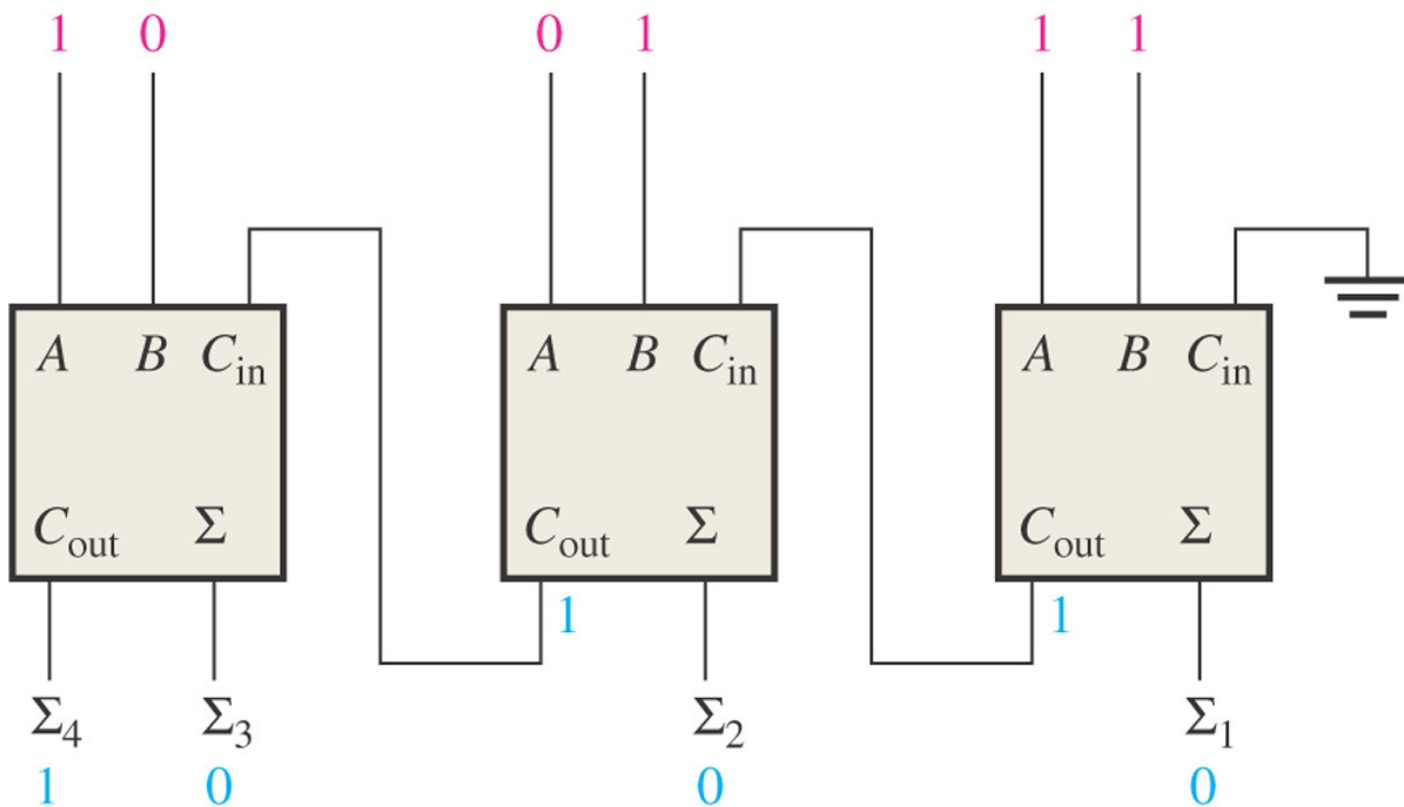
最低有效位可使用半加器或进位接地的全加器

使用两个全加器组成一个多位加法器



加法器

- 例：3位并行加法器求和，并计算101和011相加进位

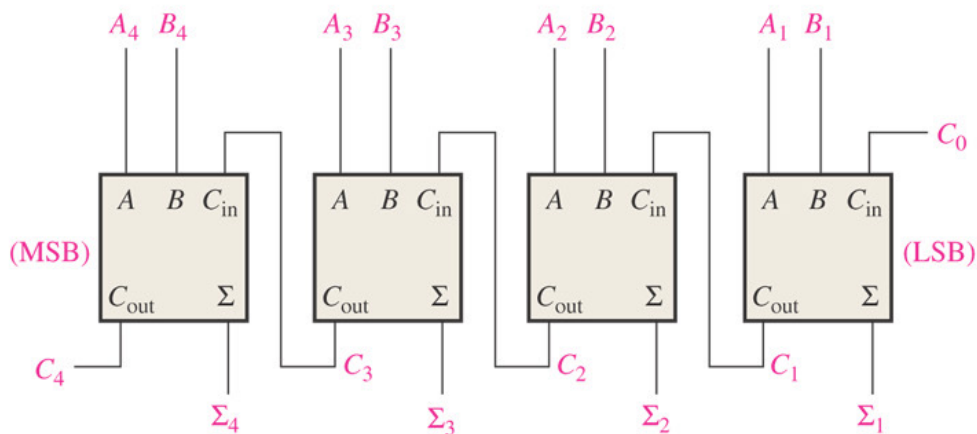




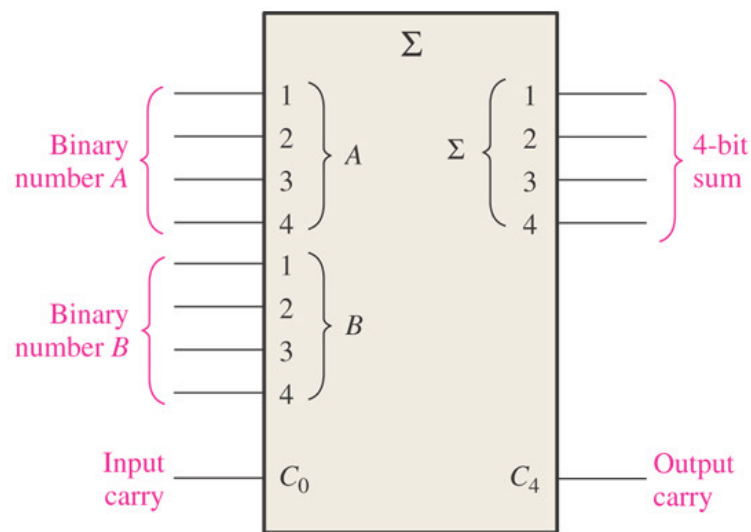
加法器

• 3. 多位加法器（并行二进制加法器）

➤ 4位并行加法器：4位一组称为1个半字节



(a) Block diagram



(b) Logic symbol

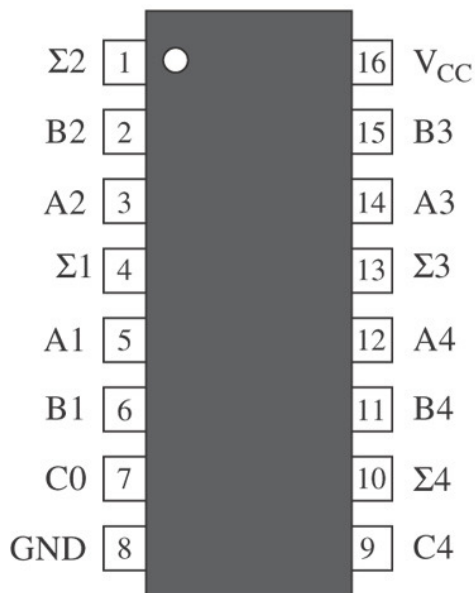


加法器

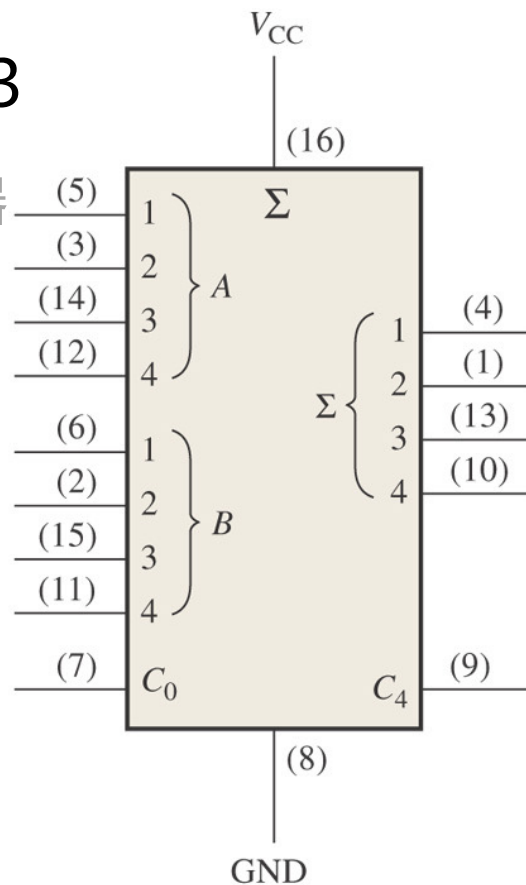
• 3. 多位加法器（并行二进制加法器）

➤ 4位并行加法器：74LS283

-283为具有超前进位的4位全加器



(a) Pin diagram of 74LS283



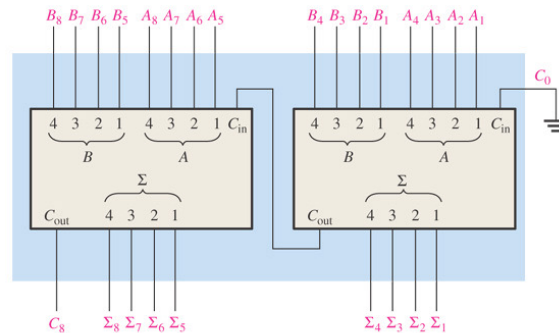
(b) 74LS283 logic symbol



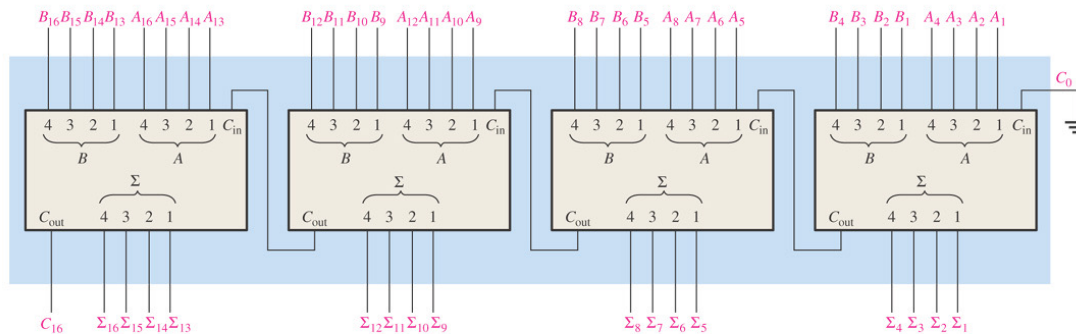
加法器

• 3. 多位加法器（并行二进制加法器）

➤ 加法器扩展：通过级联扩展到更多位



(a) Cascading of two 4-bit adders to form an 8-bit adder



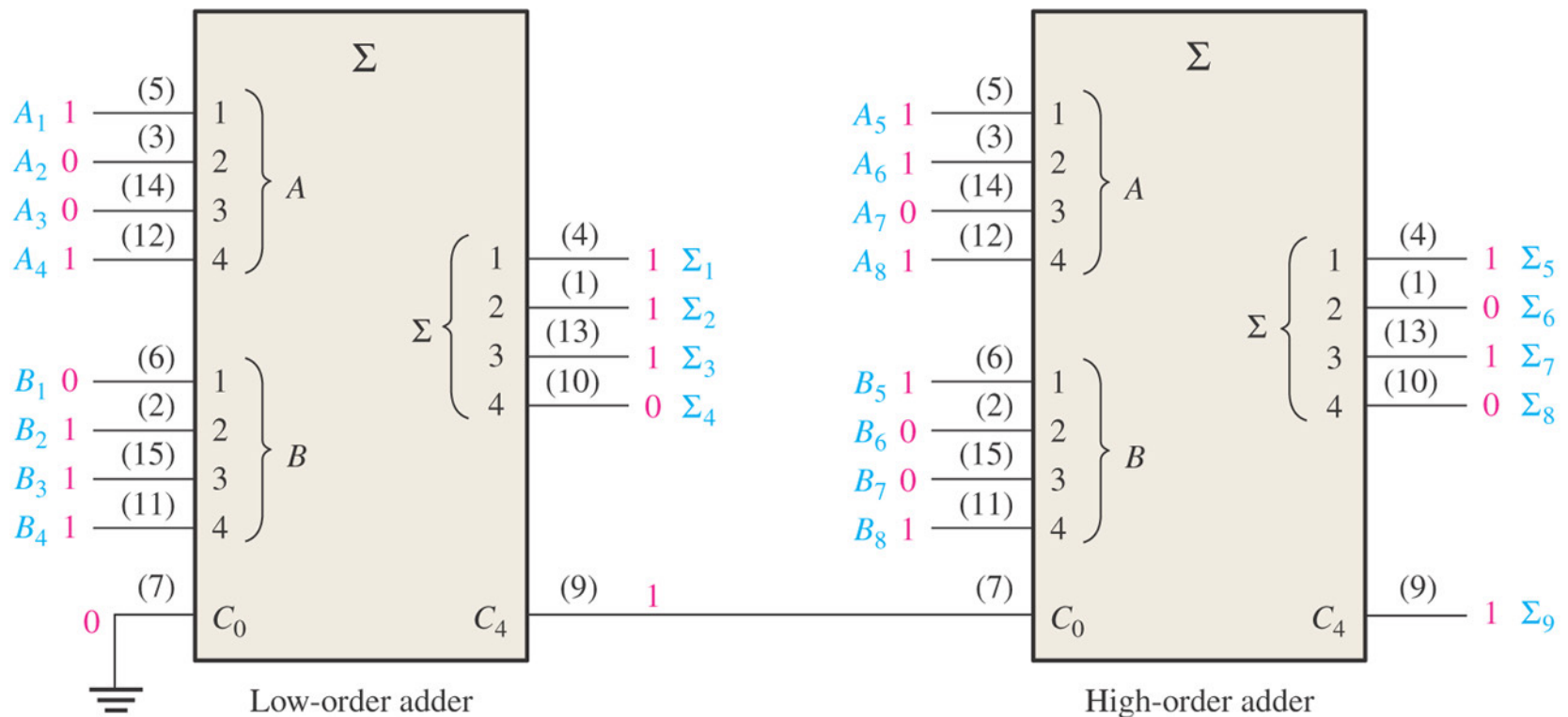
(b) Cascading of four 4-bit adders to form a 16-bit adder



加法器

- 例：2个74LS283级联为1个8位加法器，求和：

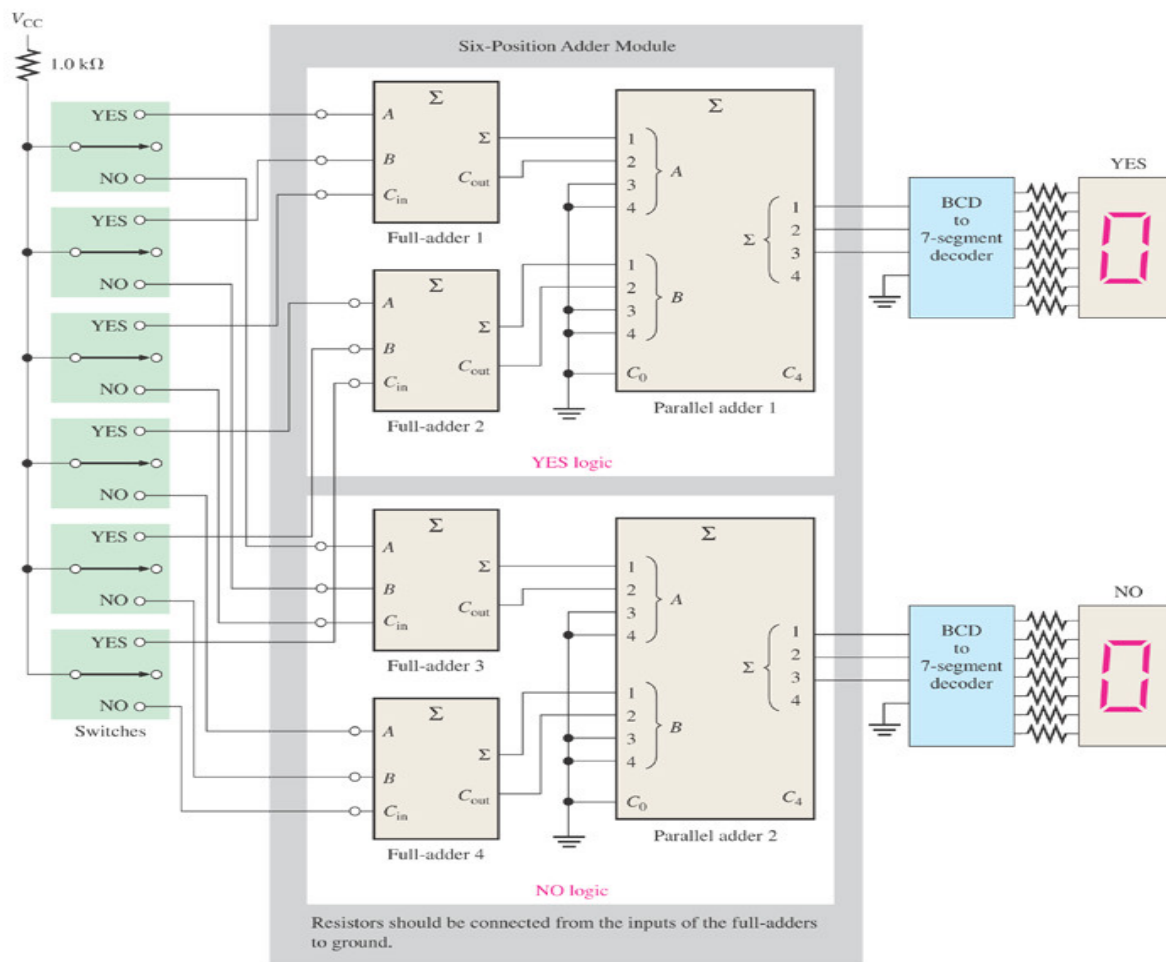
$$A_8A_7A_6A_5A_4A_3A_2A_1=10111001, \quad B_8B_7B_6B_5B_4B_3B_2B_1=10011110$$





加法器

- 应用：投票系统

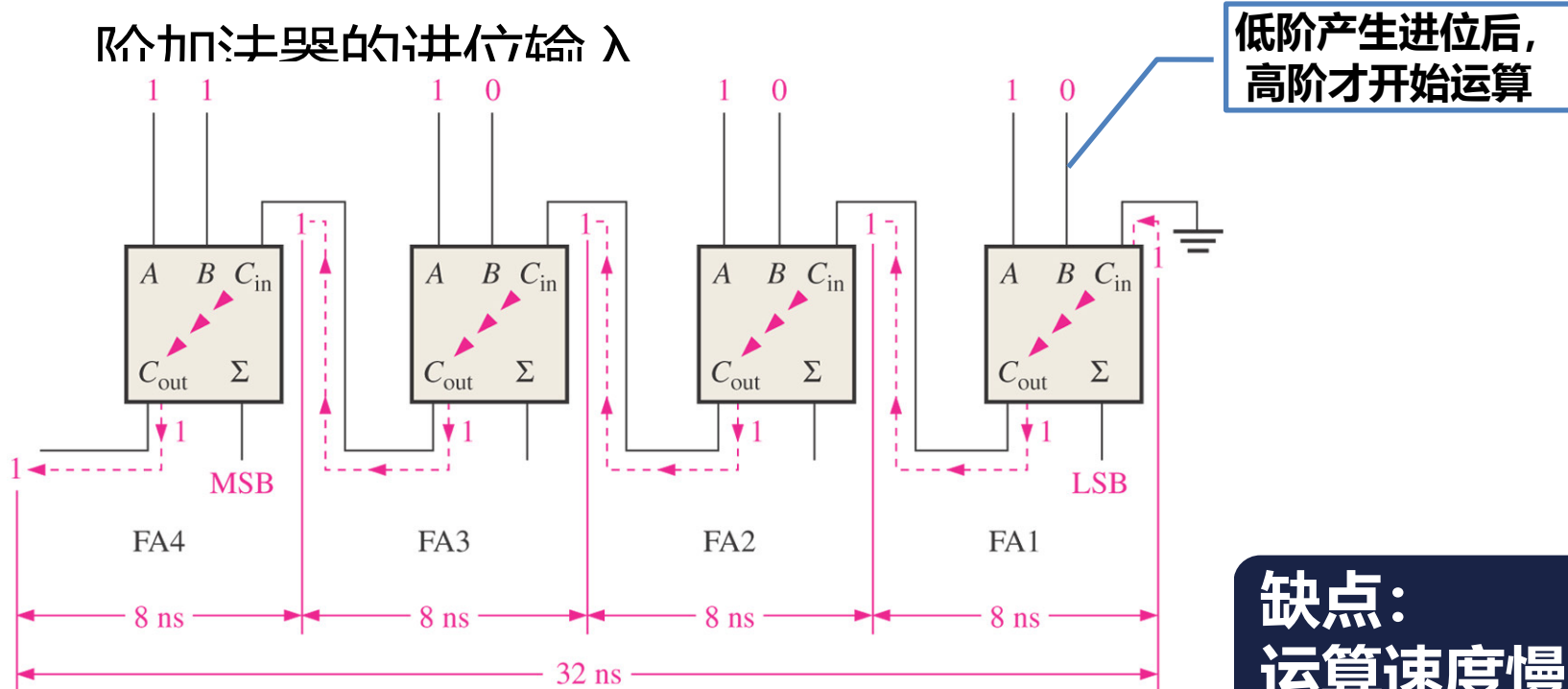




加法器

4. 异步进位与超前进位加法器

➤ 异步进位加法器：每个全法器的进位输出连接到下一个高阶加法器的进位输入



四位异步并行加法器进位输出延迟的“最坏情况”

缺点：
运算速度慢



加法器

4. 异步进位与超前进位加法器

➤ **超前进位加法器**：提前使用每一级的进位，并基于输入，通过进位生成或进位传输函数产生进位

➤ 进位生成：

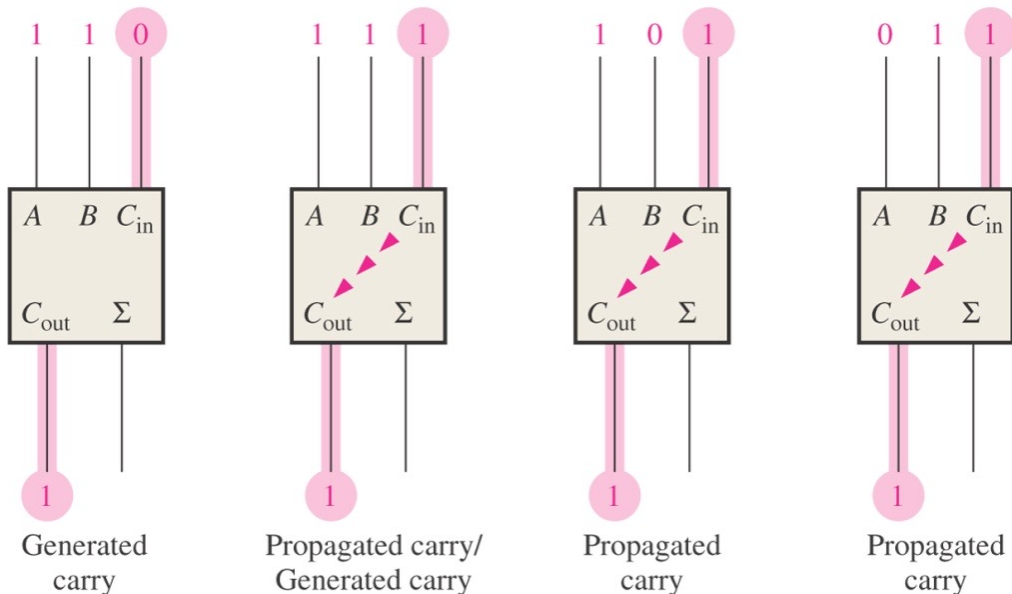
$$C_g = AB$$

➤ 进位传输：

$$C_p = A + B$$

➤ 输出进位：

$$C_{out} = C_g + C_p C_{in}$$





加法器

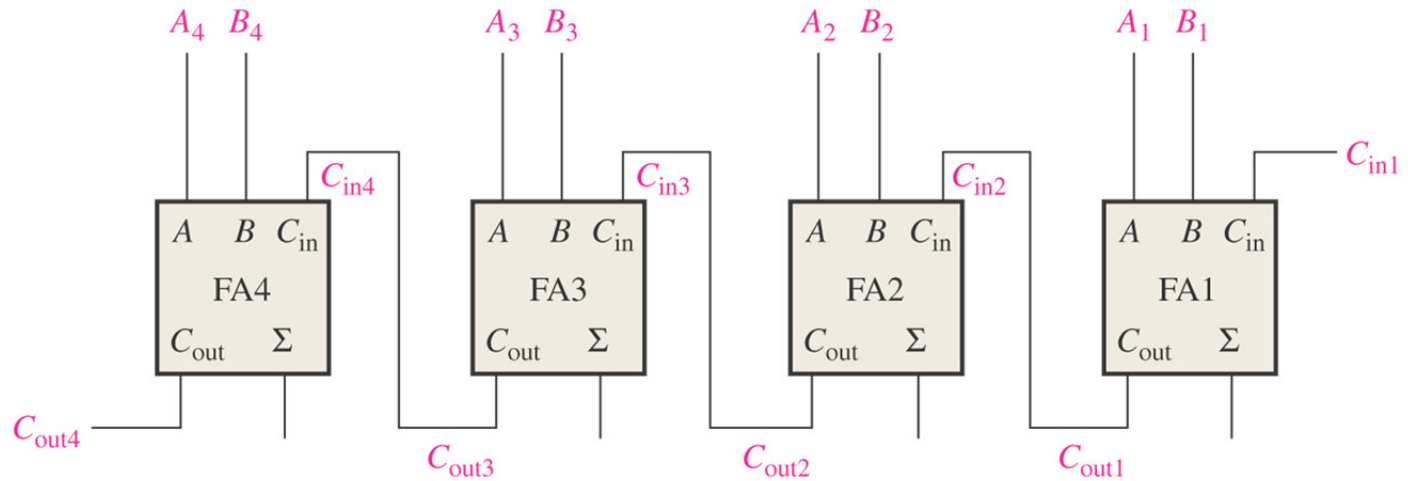
A	B	C _{in}	C _p	C _g	C _{out}
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	1	0	0
0	1	1	1	0	1
1	0	0	1	0	0
1	0	1	1	0	1
1	1	0	1	1	1
1	1	1	1	1	1

$$C_g = AB \qquad C_p = A + B$$

$$C_{out} = C_g + C_p C_{in}$$



加法器



Full-adder 4

$$C_{g4} = A_4 B_4$$

$$C_{p4} = A_4 + B_4$$

Full-adder 3

$$C_{g3} = A_3 B_3$$

$$C_{p3} = A_3 + B_3$$

Full-adder 2

$$C_{g2} = A_2 B_2$$

$$C_{p2} = A_2 + B_2$$

Full-adder 1

$$C_{g1} = A_1 B_1$$

$$C_{p1} = A_1 + B_1$$

$$C_{out1} = C_{g1} + C_{p1} C_{in1}$$

$$C_{in2} = C_{out1}$$

$$C_{out2} = C_{g2} + C_{p2} C_{in2} = C_{g2} + C_{p2} C_{out1} = C_{g2} + C_{p2} (C_{g1} + C_{p1} C_{in1})$$

$$C_{in3} = C_{out2}$$

.....

$$C_{out3} = C_{g3} + C_{p3} C_{in3} = C_{g3} + C_{p3} C_{out2} = C_{g3} + C_{p3} (C_{g2} + C_{p2} (C_{g1} + C_{p1} C_{in1}))$$



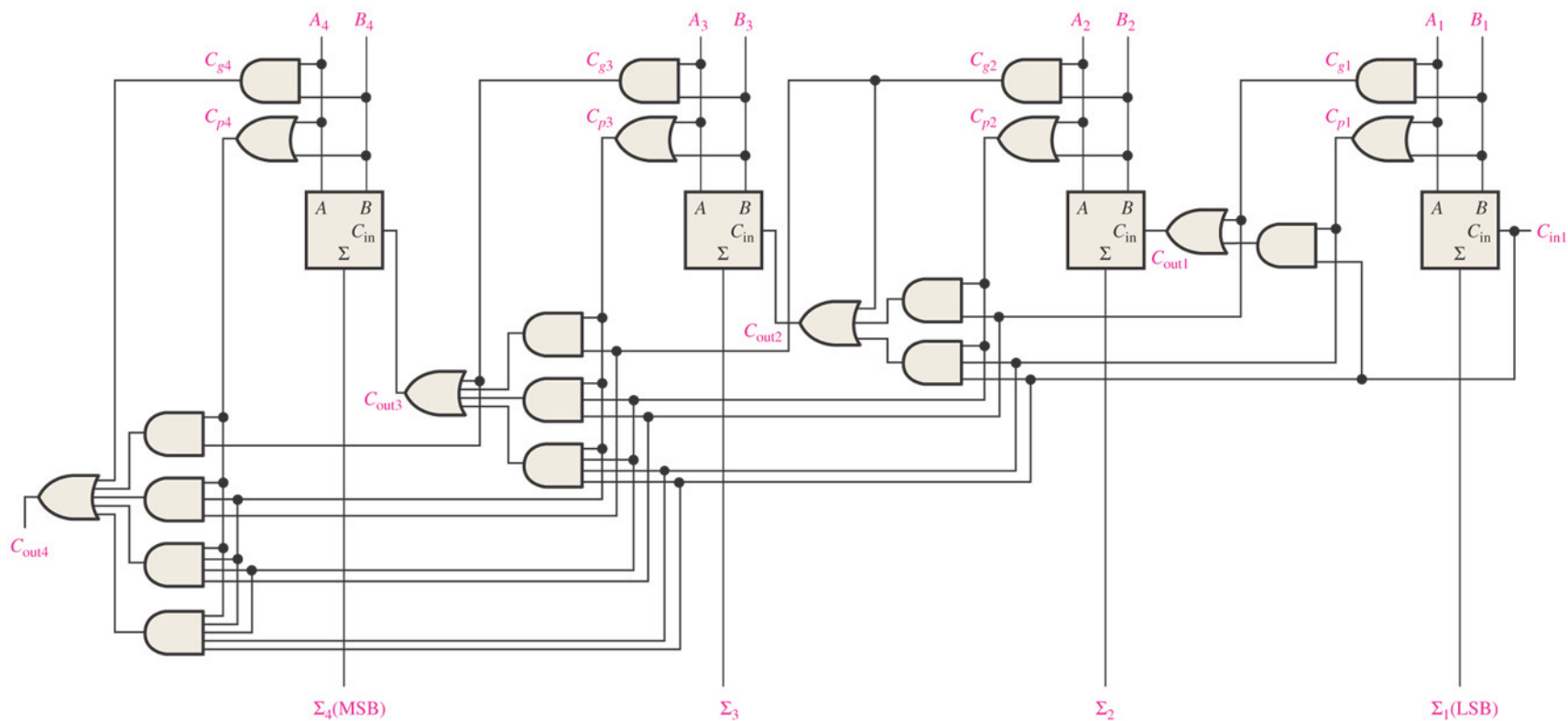
加法器

• 4. 异步进位与超前进位加法器

- **超前进位加法器**：每一级全加器的输出进位仅由最初的输入进位 (C_{in1})、那一级的函数 C_g 和 C_p 以及前一级函数的 C_g 和 C_p 确定；
- **优点**：运算速度提高；
- **缺点**：电路复杂程度随位数的增加急剧上升、可扩展性差；
- 4位加法器74LS283是一个超前进位加法器，多个级联时在中间产生异步进位。



加法器



4位超前进位加法器的逻辑框图

主要内容

1 加法器

2 比较器

3 译码器

4 编码器

5 多路复用/分配器



比较器

• 1位数值比较器

➤ 两个1位二进制数A和B比较 “是否相等”





比较器

• 1位数值比较器

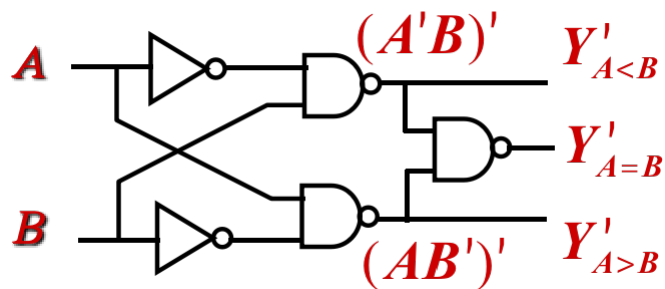
➤ 两个1位二进制数A和B “比大小”

<i>A</i>	<i>B</i>	$Y_{A>B}$	$Y_{A<B}$	$Y_{A=B}$
0	0	0	0	1
0	1	0	1	0
1	0	1	0	0
1	1	0	0	1

$$Y_{A>B} = AB'$$

$$Y_{A=B} = AB + A'B'$$

$$Y_{A<B} = A'B$$



$$Y'_{A>B} = (AB')'$$

$$Y'_{A<B} = (A'B)'$$

$$Y'_{A=B} = (AB + A'B')' = ((AB') \cdot (A'B))'$$

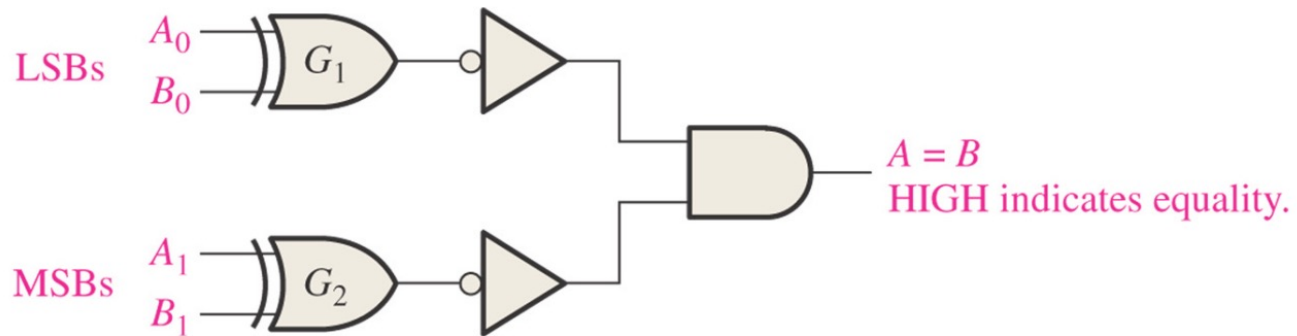
与非门实现一位大小比较器



比较器

• 多位数值比较器

➤ 两个2位二进制数A和B比较**是否相等**



General format: Binary number $A \rightarrow A_1A_0$
Binary number $B \rightarrow B_1B_0$



比较器

• 多位数值比较器

- 比较两个多位数的大小时，必须自高而低地逐位比较，而且只有在高位相等时，才需要比较低位。

比较两个 4 位数 $A = A_3 A_2 A_1 A_0$ 和 $B = B_3 B_2 B_1 B_0$ 时，

若 $A_3 > B_3$ ，则肯定是 $A > B$ 。

若 $A_3 < B_3$ ，则肯定是 $A < B$ 。

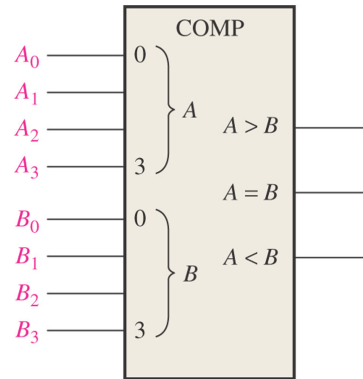
若 $A_3 = B_3$ ，则比较 A_2 和 B_2 ，依此类推。

只有当 $A_3 = B_3$ ， $A_2 = B_2$ ， $A_1 = B_1$ ， $A_0 = B_0$ 时，

才有 $A = B$ 。



比较器



Compare $A_3A_2A_1A_0$ with $B_3B_2B_1B_0$

$$Y_{(A<B)} = \overline{A_3}B_3 + \overline{(A_3 \oplus B_3)}A_2B_2 + \overline{(A_3 \oplus B_3)}\overline{(A_2 \oplus B_2)}A_1B_1 \\ + \overline{(A_3 \oplus B_3)}\overline{(A_2 \oplus B_2)}\overline{(A_1 \oplus B_1)}A_0B_0$$

$$Y_{(A=B)} = \overline{(A_3 \oplus B_3)}\overline{(A_2 \oplus B_2)}\overline{(A_1 \oplus B_1)}\overline{(A_0 \oplus B_0)}$$

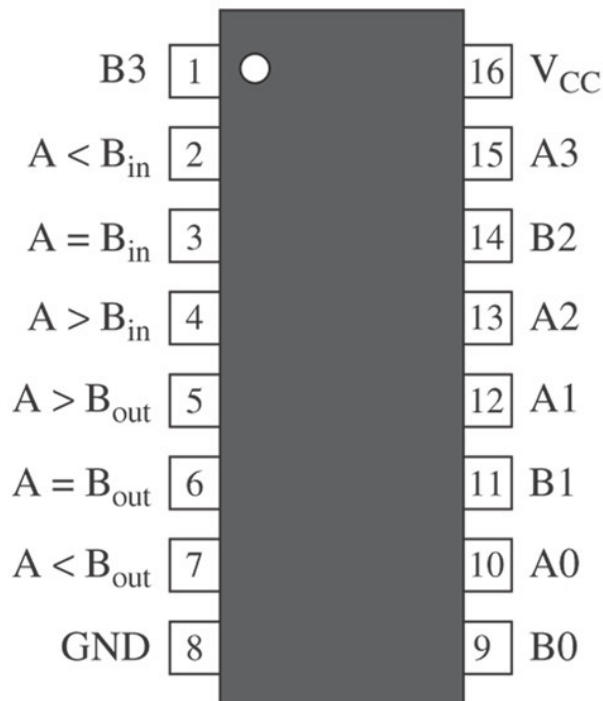
$$Y_{(A>B)} = \overline{(Y_{(A<B)} + Y_{(A=B)})}$$



比较器

• 多位数值比较器

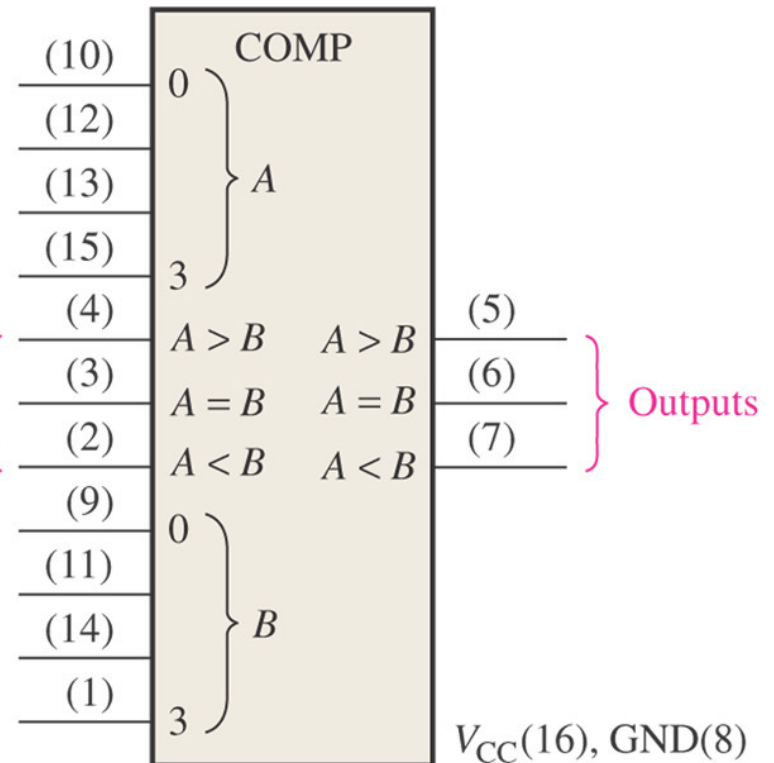
➤ 74LS85四位大小比较器



(a) Pin diagram

级联输入

Cascading inputs

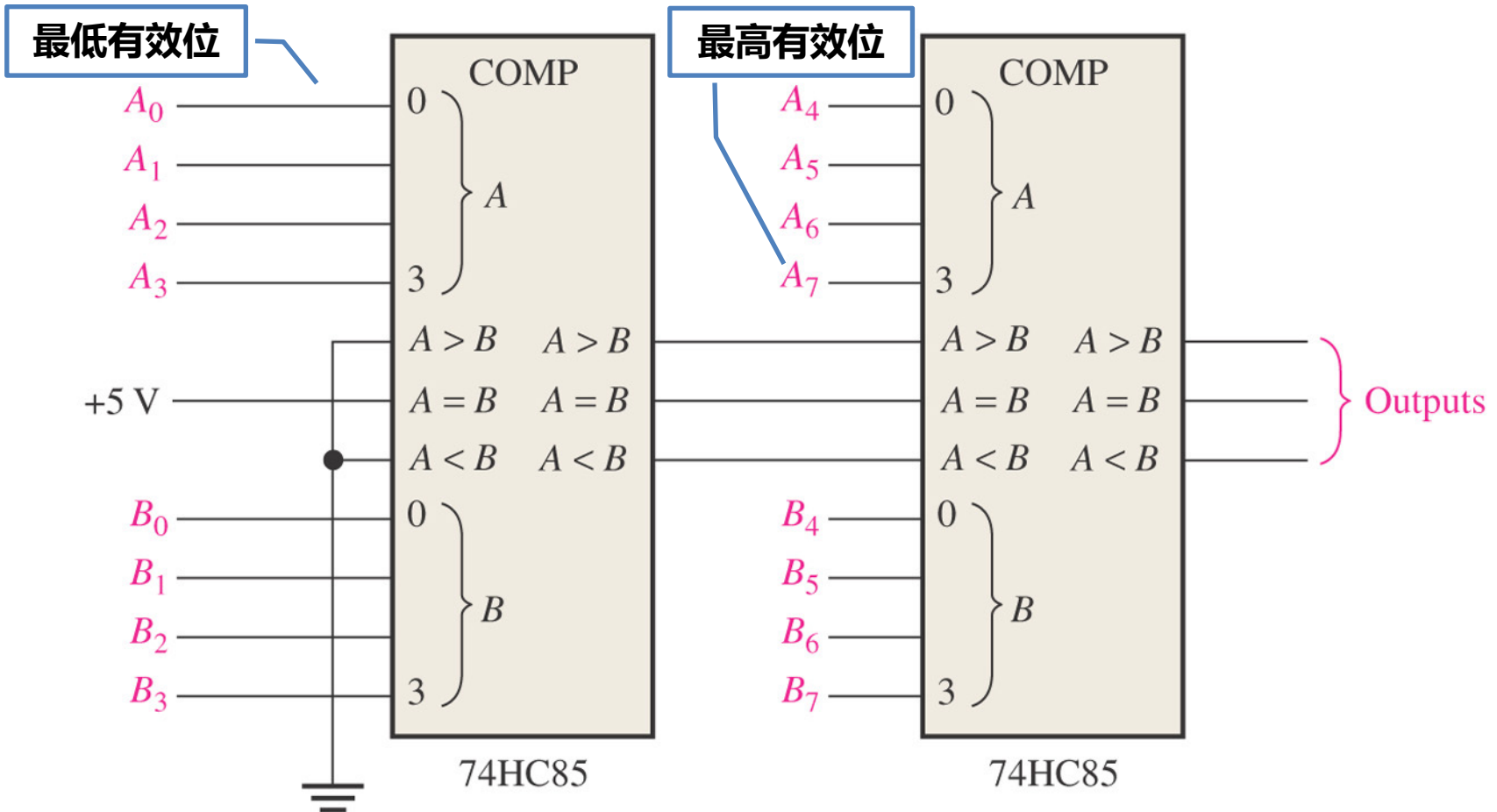


(b) Logic symbol



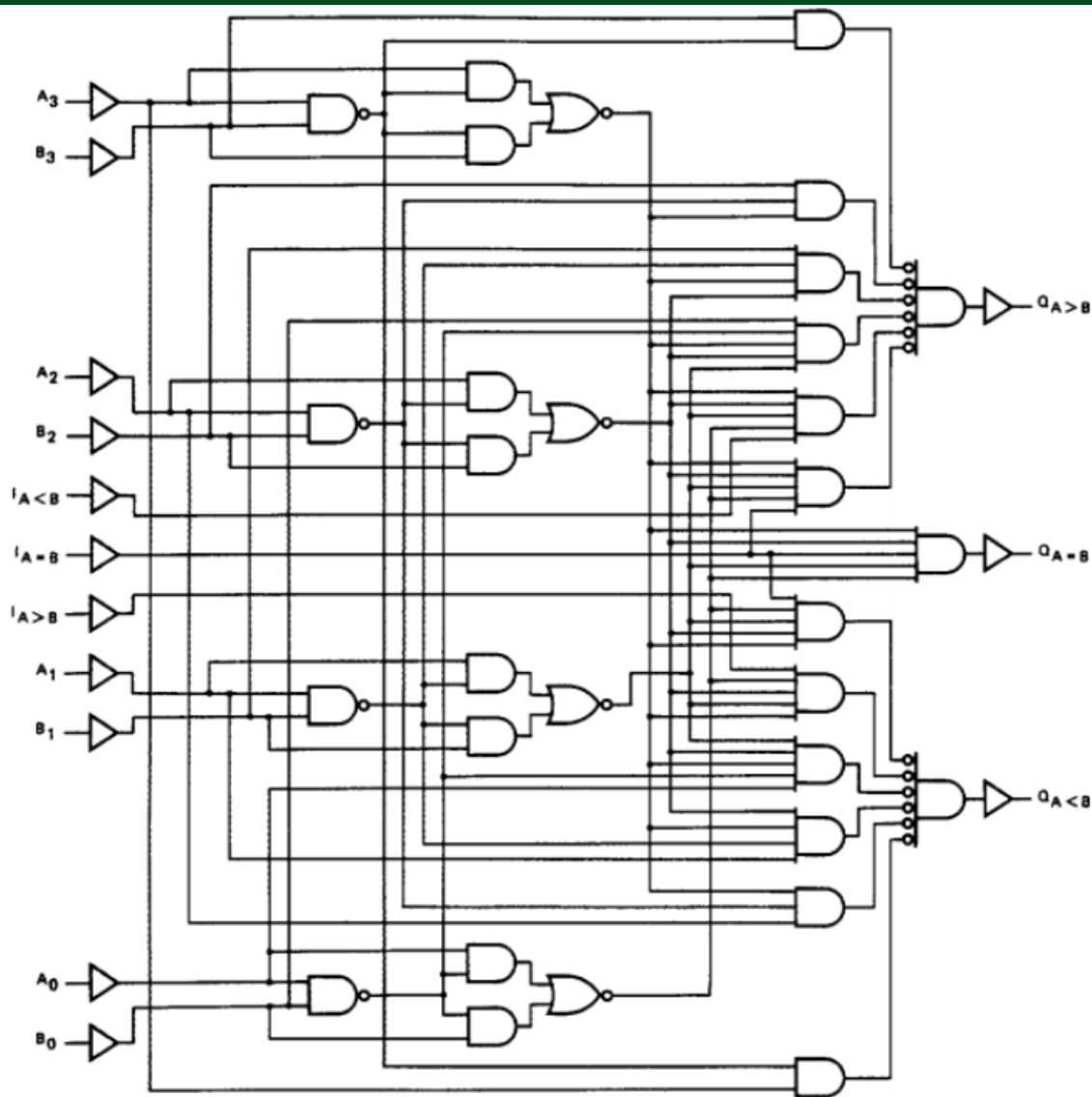
比较器

➤ 使用两个74LS85的八位大小比较器





比较器



主要内容

1 加法器

2 比较器

3 译码器

4 编码器

5 多路复用/分配器



译码器

- 译码器

- 功能：是将每个输入的二进制代码，译成对应的输出高、低电平信号。
- 常用的译码器电路有三大类：
 - 二进制译码器
 - 二 - 十进制译码器
 - 显示译码器



译码器

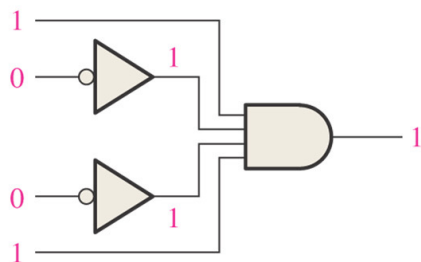
• 1. 二进制译码器

- 输入是一组二进制代码，输出是一组与输入代码一一对应的高、低电平信号。
- 检测其输入上是否存在指定的位组合
- 按指定的输出级别指示该代码的存在
- 具有 n 条输入线和 2^n 条输出线

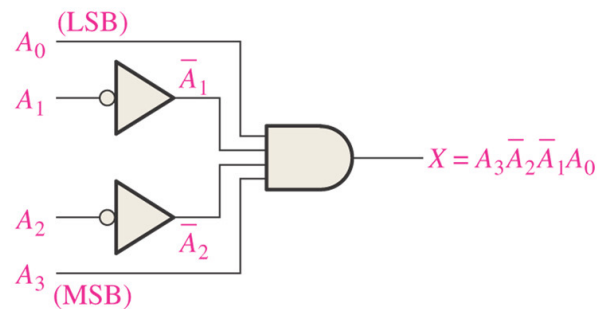


译码器

• 1. 二进制译码器

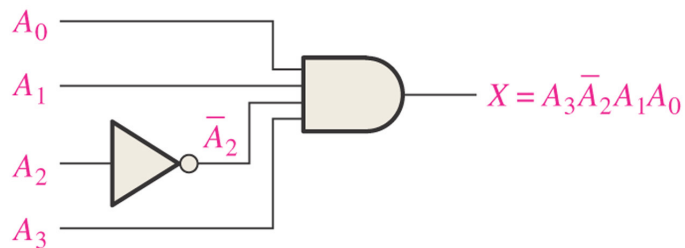


(a)



(b)

高电平输出有效的二进制数**1001**的译码逻辑



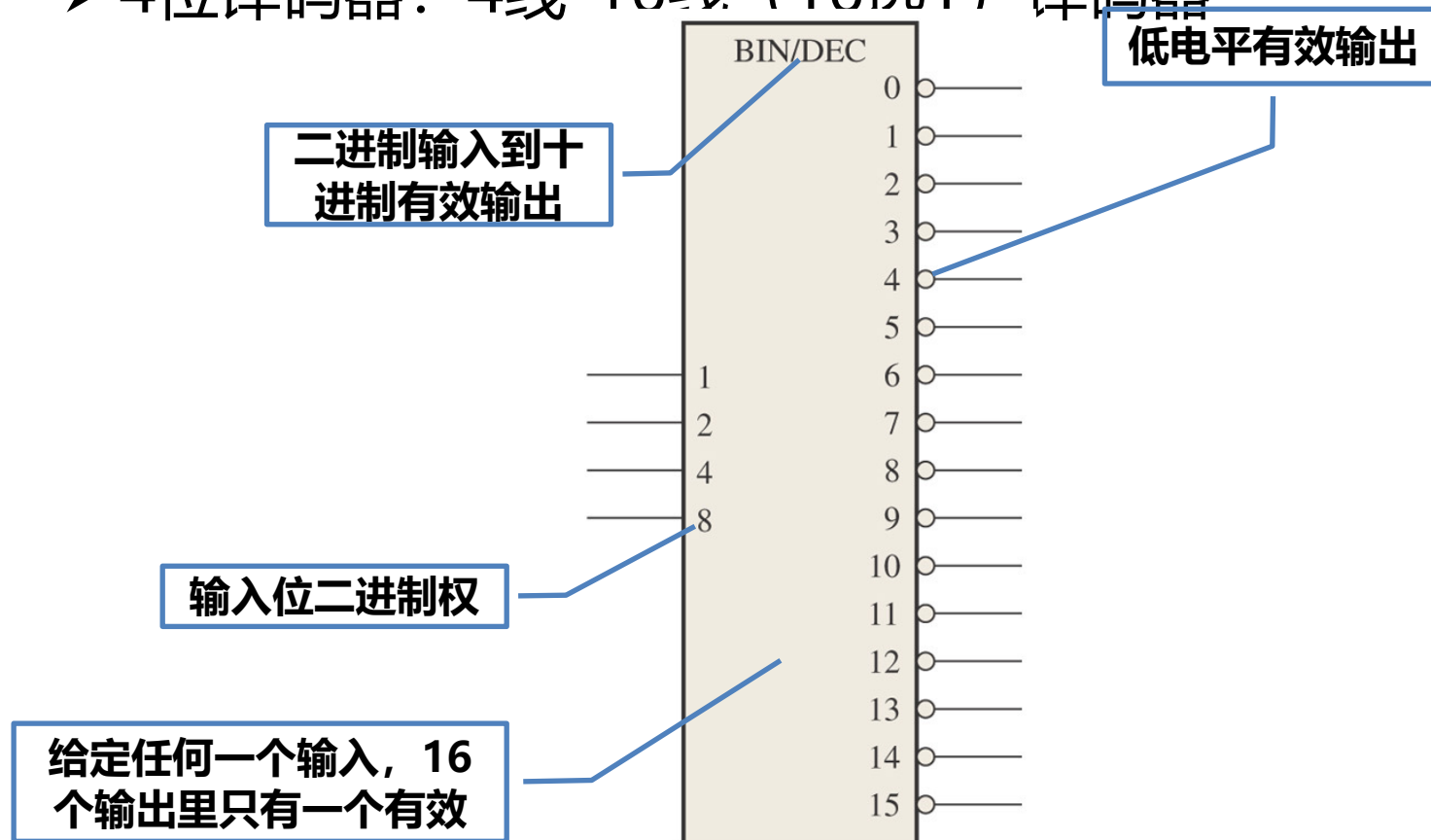
输入为**1011**时产生一个高电平输出的译码逻辑电路



译码器

• 1. 二进制译码器

➤ 4位译码器：4线-16线（16选1）译码器





译码器

➤ 4位译码器真值表

每个输出对应一个最小项

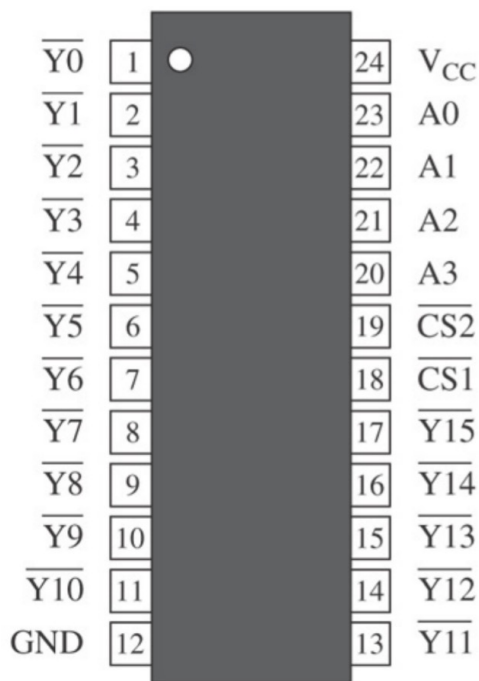
DECIMAL DIGIT	BINARY INPUTS				DECODING FUNCTION	OUTPUTS															
	A ₃	A ₂	A ₁	A ₀		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	$\overline{A_3}\overline{A_2}\overline{A_1}\overline{A_0}$	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	0	0	0	1	$\overline{A_3}\overline{A_2}\overline{A_1}A_0$	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	0	0	1	0	$\overline{A_3}\overline{A_2}A_1\overline{A_0}$	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1
3	0	0	1	1	$\overline{A_3}\overline{A_2}A_1A_0$	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1
4	0	1	0	0	$\overline{A_3}A_2\overline{A_1}\overline{A_0}$	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1
5	0	1	0	1	$\overline{A_3}A_2\overline{A_1}A_0$	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1
6	0	1	1	0	$\overline{A_3}A_2A_1\overline{A_0}$	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1
7	0	1	1	1	$\overline{A_3}A_2A_1A_0$	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1
8	1	0	0	0	$A_3\overline{A_2}\overline{A_1}\overline{A_0}$	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1
9	1	0	0	1	$A_3\overline{A_2}\overline{A_1}A_0$	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1
10	1	0	1	0	$A_3\overline{A_2}A_1\overline{A_0}$	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1
11	1	0	1	1	$A_3\overline{A_2}A_1A_0$	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1
12	1	1	0	0	$A_3A_2\overline{A_1}\overline{A_0}$	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1
13	1	1	0	1	$A_3A_2\overline{A_1}A_0$	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1
14	1	1	1	0	$A_3A_2A_1\overline{A_0}$	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1
15	1	1	1	1	$A_3A_2A_1A_0$	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0



译码器

• 1. 二进制译码器

➤ 74HC154

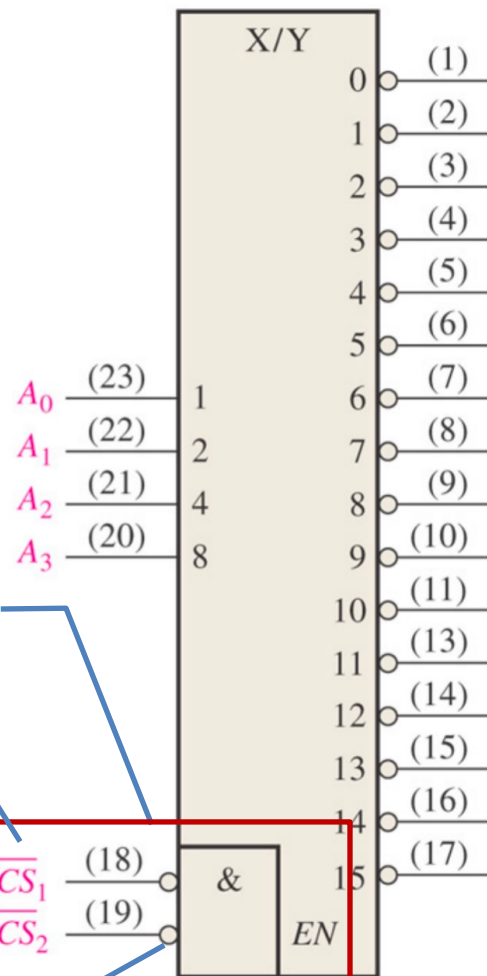


引脚图

使能函数EN：非-与门

若不是两个低电平输入，
译码器 (A_0 - A_3) 无论输入什么，输出端都是高电平（因低电平输出有效）

两个低电平有效输入，才能输出高电平激活电路

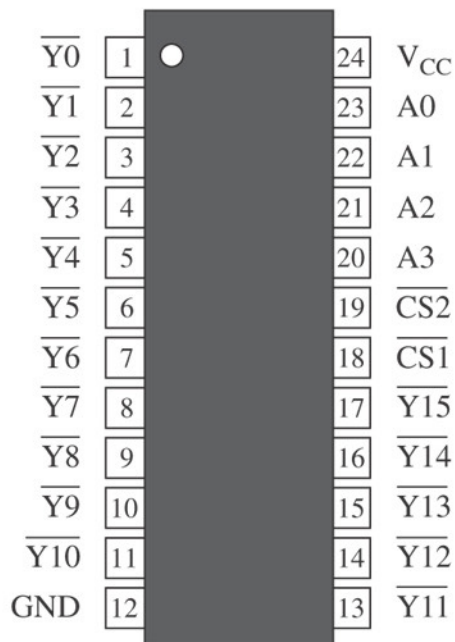


逻辑符号

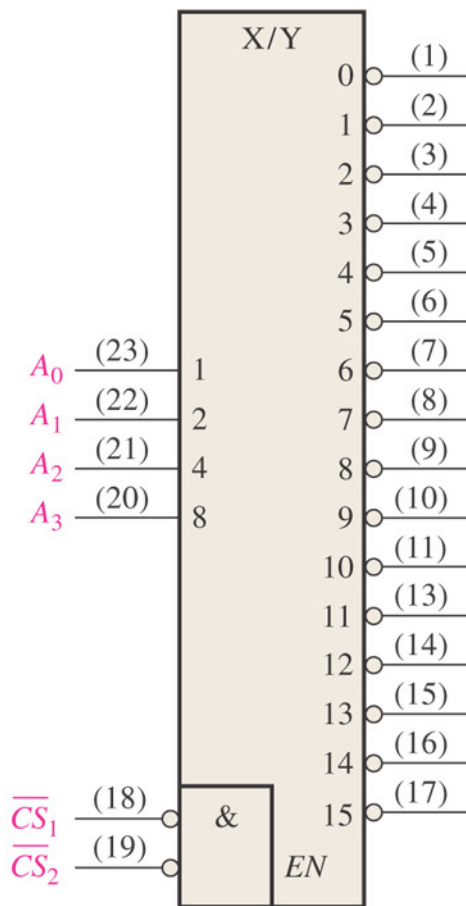


译码器

- 思考：用74HC154实现五位二进制数译码器



(a) Pin diagram

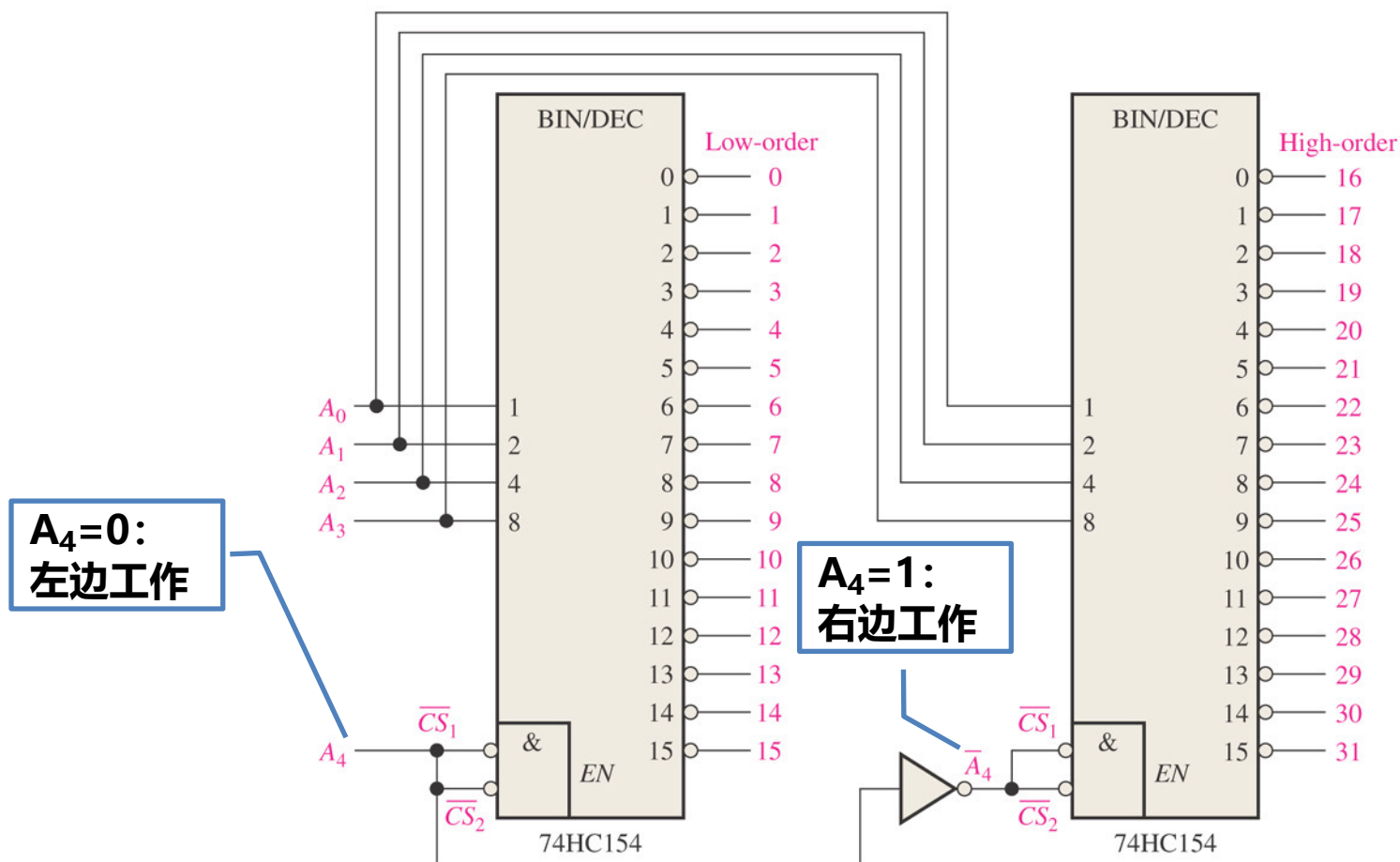


(b) Logic symbol



译码器

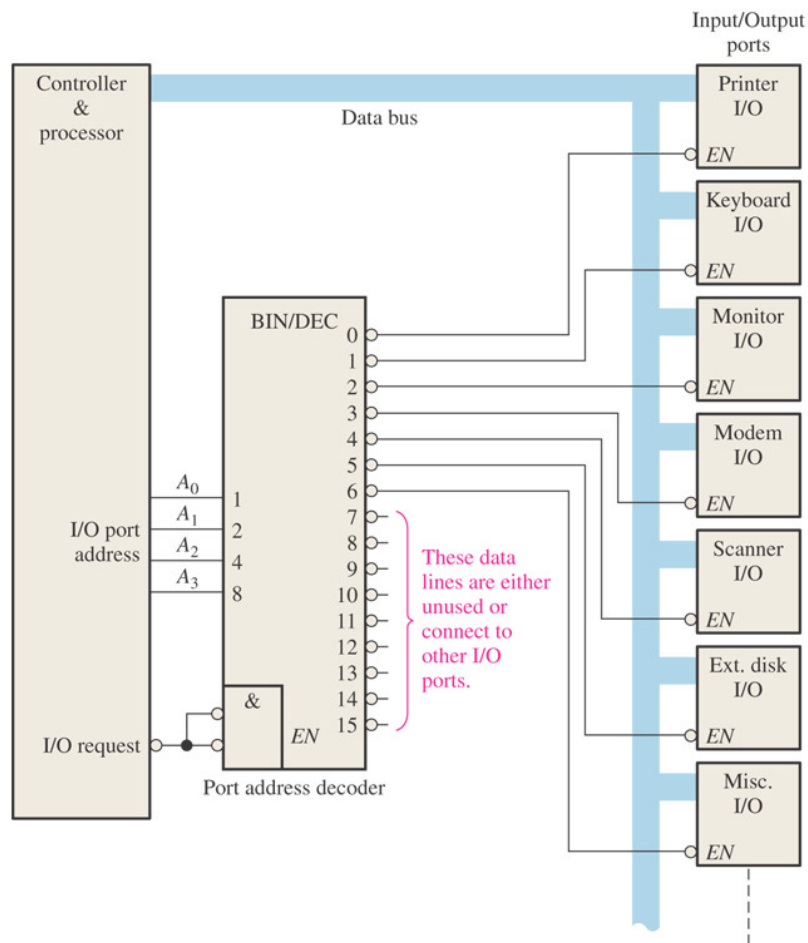
- 思考：用74HC154实现五位二进制数译码器





译码器

- 应用：4条地址线的端口地址译码器





译码器

• 2. 二 – 十进制译码器

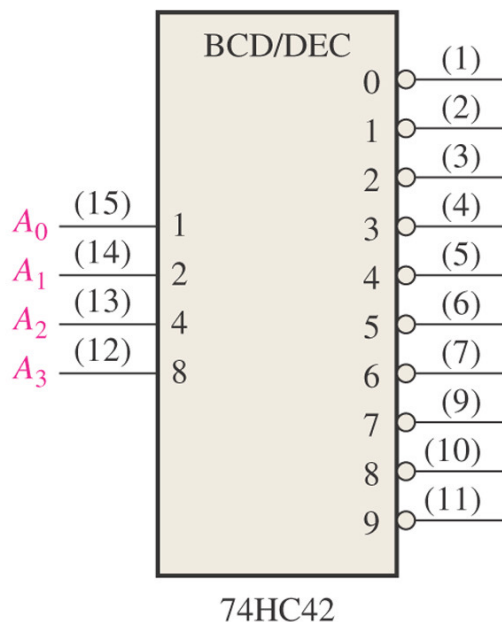
- 把每一个BCD码（8421码）转化成10个十进制数中的一个表示输出；
- 与16选一译码器相同，但只需要10个译码门
- 译码函数由与非门实现，提供低电平有效输出



译码器

• 2. 二 – 十进制译码器

➤ 74HC42



序号	输 入				输 出									
	A_3	A_2	A_1	A_0	Y'_0	Y'_1	Y'_2	Y'_3	Y'_4	Y'_5	Y'_6	Y'_7	Y'_8	Y'_9
0	0	0	0	0	0	1	1	1	1	1	1	1	1	1
1	0	0	0	1	1	0	1	1	1	1	1	1	1	1
2	0	0	1	0	1	1	0	1	1	1	1	1	1	1
3	0	0	1	1	1	1	1	0	1	1	1	1	1	1
4	0	1	0	0	1	1	1	1	0	1	1	1	1	1
5	0	1	0	1	1	1	1	1	1	0	1	1	1	1
6	0	1	1	0	1	1	1	1	1	1	0	1	1	1
7	0	1	1	1	1	1	1	1	1	1	1	0	1	1
8	1	0	0	0	1	1	1	1	1	1	1	1	0	1
9	1	0	0	1	1	1	1	1	1	1	1	1	1	0
伪 码	1	0	1	0	1	1	1	1	1	1	1	1	1	1
	1	0	1	1	1	1	1	1	1	1	1	1	1	1
	1	1	0	0	1	1	1	1	1	1	1	1	1	1
	1	1	0	1	1	1	1	1	1	1	1	1	1	1
	1	1	1	0	1	1	1	1	1	1	1	1	1	1
	1	1	1	1	1	1	1	1	1	1	1	1	1	1

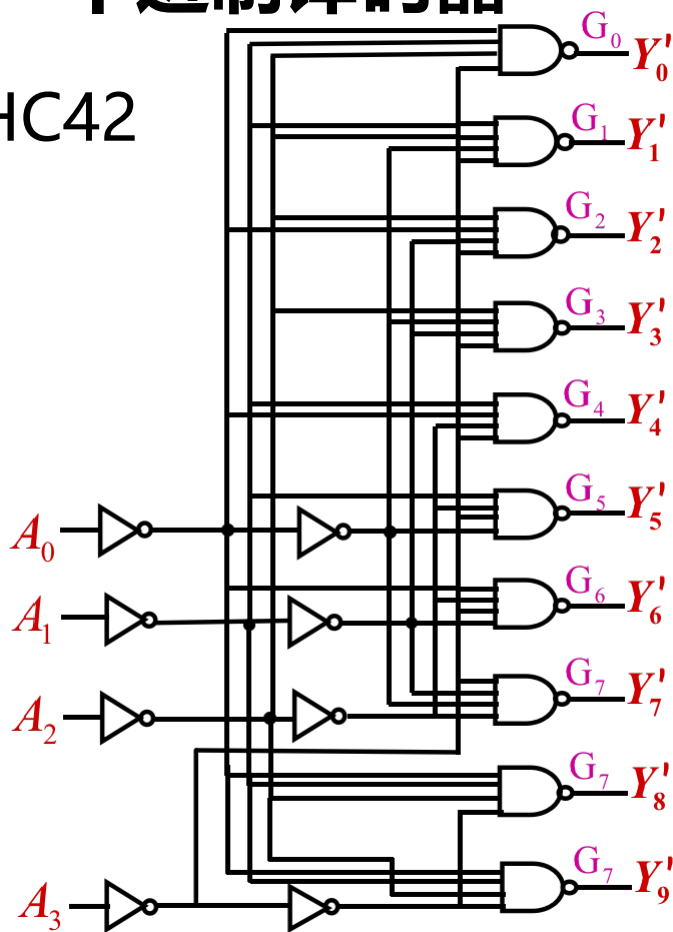
74HC42真值表



译码器

• 2. 二 - 十进制译码器

➤ 74HC42



74HC42逻辑图

$$Y'_0 = (A'_3 A'_2 A'_1 A'_0)'$$

$$Y'_1 = (A'_3 A'_2 A'_1 A'_0)'$$

$$Y'_2 = (A'_3 A'_2 A'_1 A'_0)'$$

$$Y'_3 = (A'_3 A'_2 A'_1 A'_0)'$$

$$Y'_4 = (A'_3 A'_2 A'_1 A'_0)'$$

$$Y'_5 = (A'_3 A'_2 A'_1 A'_0)'$$

$$Y'_6 = (A'_3 A'_2 A'_1 A'_0)'$$

$$Y'_7 = (A'_3 A'_2 A'_1 A'_0)'$$

$$Y'_8 = (A'_3 A'_2 A'_1 A'_0)'$$

$$Y'_9 = (A'_3 A'_2 A'_1 A'_0)'$$

74HC42输出函数表达式



译码器

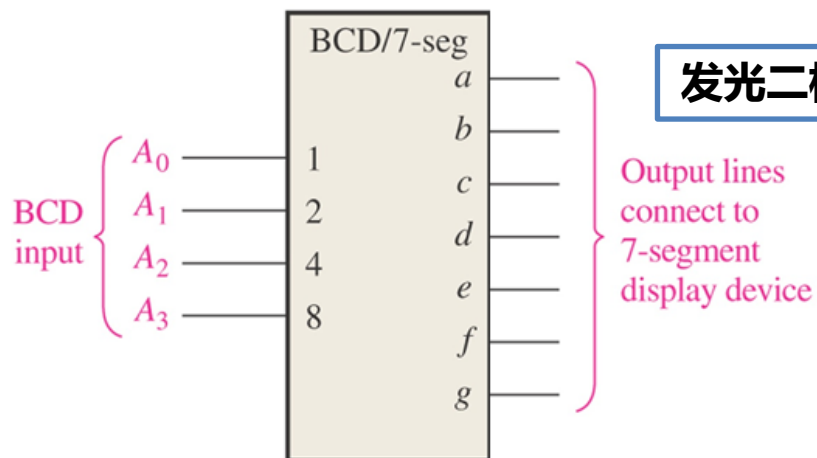
• 3. 显示译码器 (BCD-7段译码器)

- 为了能以十进制数码直观地显示数字系统的运行数据,目前被广泛应用。
- 常见的七段字符显示器有: **液晶显示器**和**半导体数码管**。

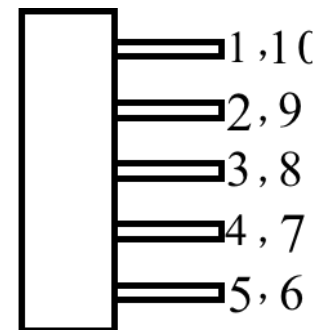
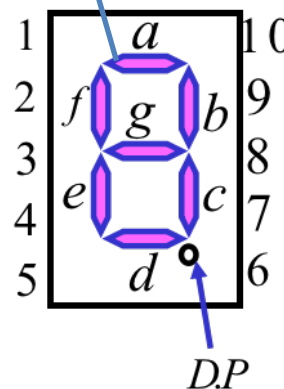
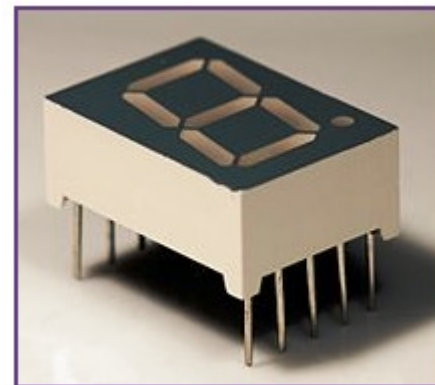


译码器

• 半导体数码管



发光二极管LED

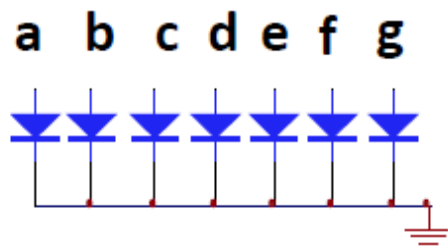


优点：工作电压低、体积小、寿命长、可靠性高、响应时间短、亮度较高。
缺点：工作电流较大。



译码器

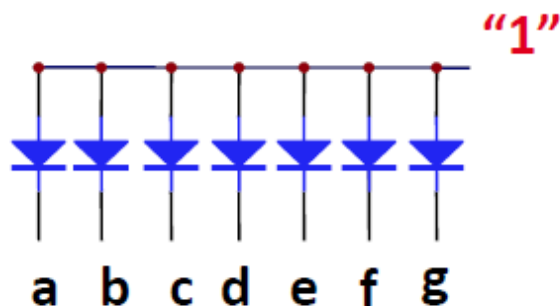
Connection: 连接方式不同分成**共阴极**和**共阳极**两种



BS201A

Common-cathode (共阴极)

diodes $\longrightarrow$ logic **high** $\longrightarrow$ bright



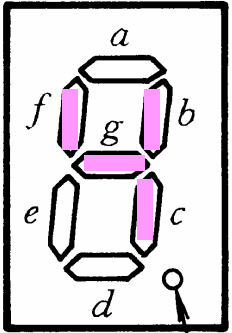
BS201B

Common-anode (共阳极)

diodes $\longrightarrow$ logic **low** $\longrightarrow$ bright



译码器

输 入					输 出							字形
数字	A_3	A_2	A_1	A_0	Y_a	Y_b	Y_c	Y_d	Y_e	Y_f	Y_g	
0	0	0	0	0	1	1	1	1	1	1	0	
1	0	0	0	1	0	1	1	0	0	0	0	
2	0	0	1	0	1	1	0	1	1	0	1	
3	0	0	1	1	1	1	1	1	0	0	1	
4	0	1	0	0	0	1	1	0	0	1	1	
5	0	1	0	1	1	0	1	1	0	1	1	
6	0	1	1	0	0	0	1	1	1	1	1	
7	0	1	1	1	1	1	1	0	0	0	0	
8	1	0	0	0	1	1	1	1	1	1	1	
9	1	0	0	1	1	1	1	0	0	1	1	
10	1	0	1	0	0	0	0	1	1	0	1	
11	1	0	1	1	0	0	1	1	0	0	1	
12	1	1	0	0	0	1	0	0	0	1	1	
13	1	1	0	1	1	0	0	1	0	1	1	
14	1	1	1	0	0	0	0	1	1	1	1	
15	1	1	1	1	0	0	0	0	0	0	0	



译码器

		A_1A_2			
		00	01	11	10
A_3A_2	00	1	0	1	1
	01	0	1	1	0
	11	0	1	0	0
	10	1	1	0	0

(a)

		A_1A_2			
		00	01	11	10
A_3A_2	00	1	1	1	1
	01	1	0	1	0
	11	1	0	0	0
	10	1	1	0	0

(b)

		A_1A_2			
		00	01	11	10
A_3A_2	00	1	1	1	0
	01	1	1	1	1
	11	0	0	0	0
	10	1	1	1	0

(c)

		A_1A_2			
		00	01	11	10
A_3A_2	00	1	0	1	1
	01	0	1	0	1
	11	0	1	0	1
	10	1	0	1	1

(d)

		A_1A_2			
		00	01	11	10
A_3A_2	00	1	0	0	1
	01	0	0	0	1
	11	0	0	0	1
	10	1	0	0	1

(e)

		A_1A_2			
		00	01	11	10
A_3A_2	00	1	0	0	0
	01	1	1	0	1
	11	1	1	0	1
	10	1	1	0	0

(f)

		A_1A_2			
		00	01	11	10
A_3A_2	00	0	0	1	1
	01	1	1	0	1
	11	1	1	0	1
	10	1	1	1	1

(g)



译码器

$A_3 A_2 \backslash A_1 A_0$					
		00	01	11	10
$A_3 A_2$	00	1	0	1	1
	01	0	1	1	0
	11	0	1	0	0
	10	1	1	0	0

(a)

$A_3 A_2 \backslash A_1 A_0$					
		00	01	11	10
$A_3 A_2$	00	1	1	1	1
	01	1	0	1	0
	11	1	0	0	0
	10	1	1	0	0

(b)

$$Y_a = (A_3' A_2' A_1' A_0 + A_3 A_1 + A_2 A_0)'$$

$$Y_b = (A_3 A_1 + A_2 A_1 A_0' + A_2 A_1' A_0)'$$



译码器

Y_c

$A_3 A_2$ \ $A_1 A_0$	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	0	0	0	0
10	1	1	1	0

(c)

Y_d

$A_3 A_2$ \ $A_1 A_0$	00	01	11	10
00	1	0	1	1
01	0	1	0	1
11	0	1	0	1
10	1	0	1	1

(d)

$$Y_c = (A_3 A_2 + A_2' A_1 A_0)'$$

$$Y_d = (A_2 A_1 A_0 + A_2 A_1' A_0' + A_2' A_1' A_0)'$$



译码器

$A_3 A_2 \backslash A_1 A_0$					
		00	01	11	10
00	00	1	0	0	1
01	01	0	0	0	1
11	11	0	0	0	1
10	10	1	0	0	1

(e)

$A_3 A_2 \backslash A_1 A_0$					
		00	01	11	10
00	00	1	0	0	0
01	01	1	1	0	1
11	11	1	1	0	1
10	10	1	1	0	0

(f)

$A_3 A_2 \backslash A_1 A_0$					
		00	01	11	10
00	00	0	0	1	1
01	01	1	1	0	1
11	11	1	1	0	1
10	10	1	1	1	1

(g)

$$Y_e = (A_2 A_1' + A_0)'$$

$$Y_f = (A_3' A_2' A_0 + A_2' A_1 + A_1 A_0)'$$

$$Y_g = (A_3' A_2' A_1' + A_2 A_1 A_0)'$$



译码器

• 3. 显示译码器 (BCD-7段译码器)

➤ 74LS47 (共阳极) 74LS48 (共阴极)

$$Y_a = \overline{(A_3 A_2 A_1 A_0 + A_3 A_1 + A_2 A_0)}$$

$$Y_b = \overline{(A_3 A_1 + A_2 A_1 A_0 + A_2 A_1 A_0)}$$

$$Y_c = \overline{(A_3 A_2 + A_2 A_1 A_0)}$$

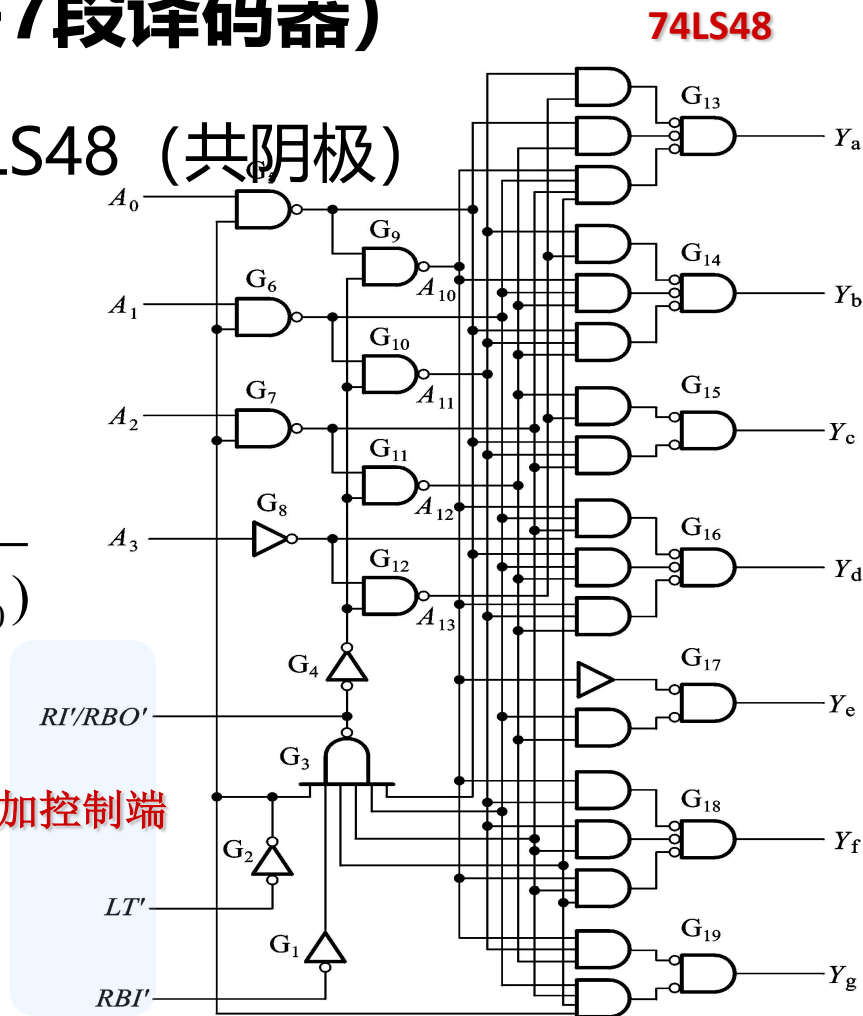
$$Y_d = \overline{(A_2 A_1 A_0 + A_2 A_1 A_0 + A_2 A_1 A_0)}$$

$$Y_e = \overline{(A_2 A_1 + A_0)}$$

$$Y_f = \overline{(A_3 A_2 A_0 + A_2 A_1 + A_1 A_0)}$$

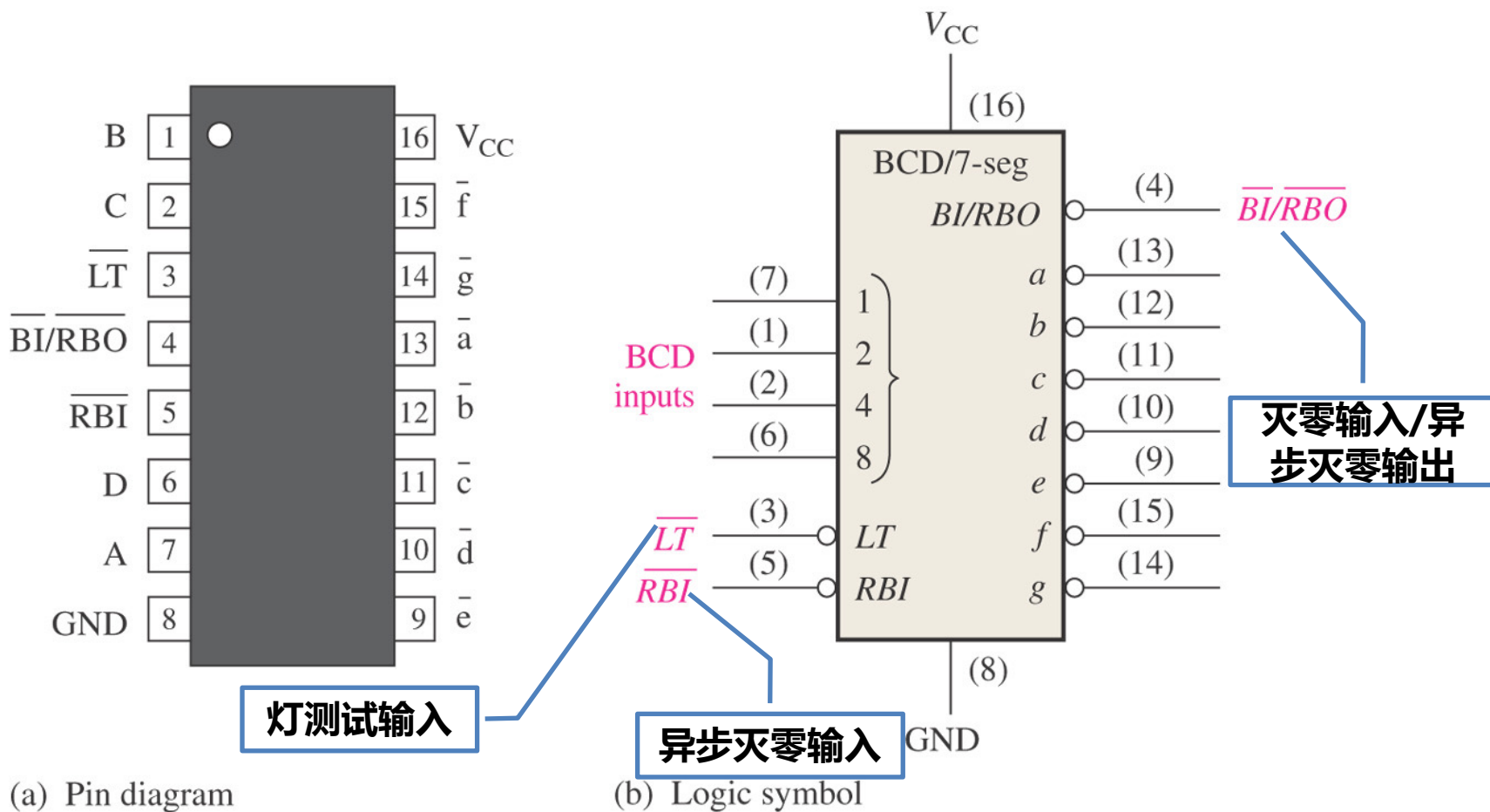
$$Y_g = \overline{(A_3 A_2 A_1 + A_2 A_1 A_0)}$$

附加控制端





译码器



74LS47引脚图和逻辑符号



译码器

• 3. 显示译码器 (BCD-7段译码器)

➤ 附加控制端的功能和用法:

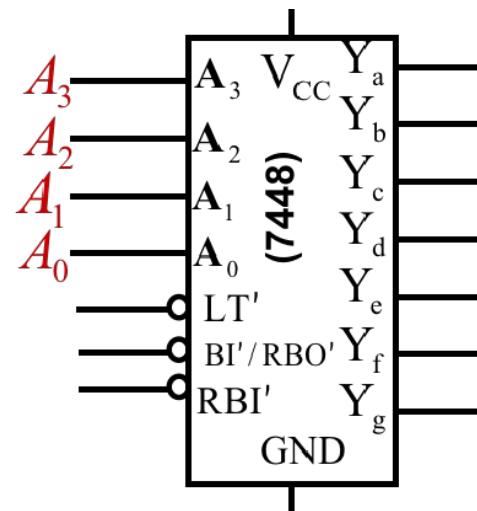
1) 灯测试: $\overline{LT}$ 输入低电平, $\overline{RBI}$ 置高时:

数码管的七段同时点亮, 可检查该数码管各段能否正常发光 (平时应置高)

2) 灭零: 将本应显示 0 但不希望显示的 0 熄灭 (03.080→3.08)

$\overline{RBI}$: 异步灭零输入,

$\overline{BI}/\overline{BRO}$: 灭灯输入/异步灭零输出





译码器

• 3. 显示译码器 (BCD-7段译码器)

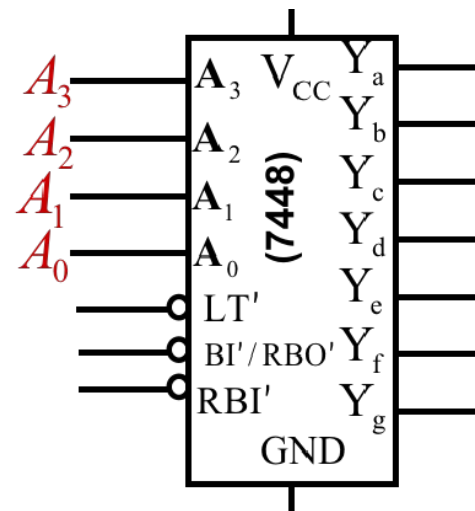
➤ 附加控制端的功能和用法:

3) $\overline{BI}/\overline{BRO}$ - 作为输入端使用时, 称为灭灯

输入控制端 $\overline{BI}$, 将被驱动的数码管熄灭。

- 作为输出端使用时, 称为灭零输出控制端。

$\overline{BRO}$: 异步灭0输出, 表示已将本该显示的零熄灭了。



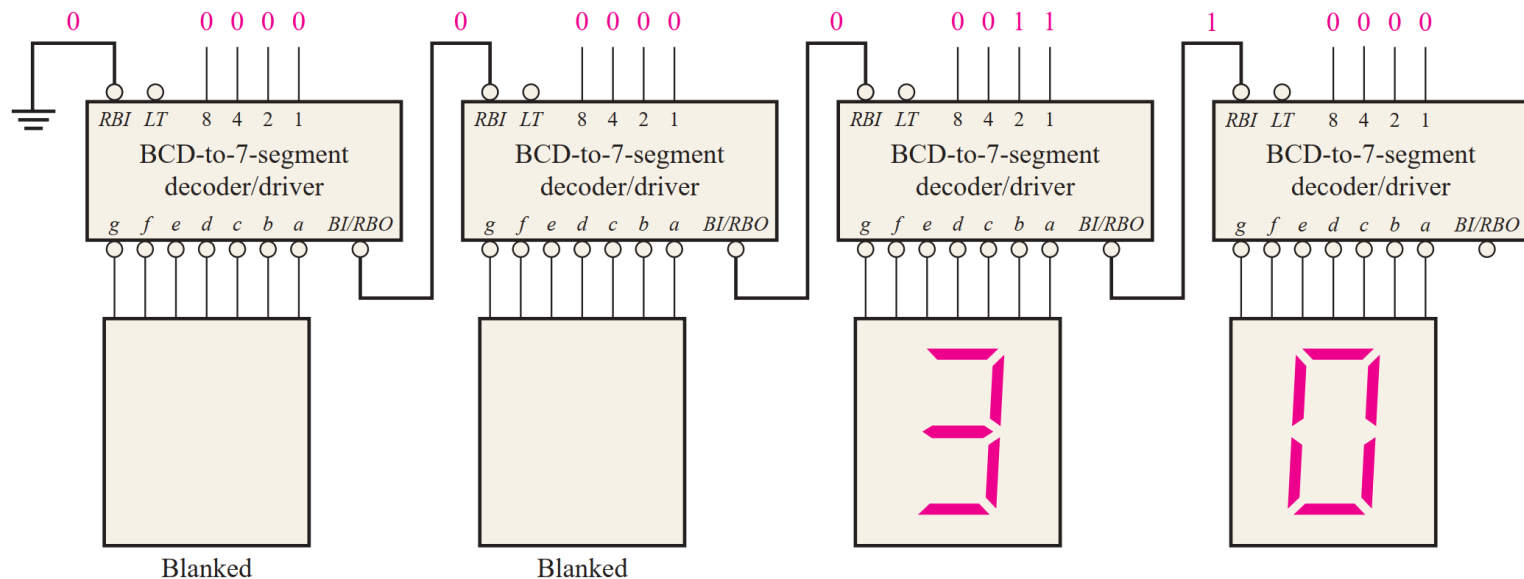


译码器

• 3. 显示译码器 (BCD-7段译码器)

➤ 附加控制端的功能和用法:

头部灭灯



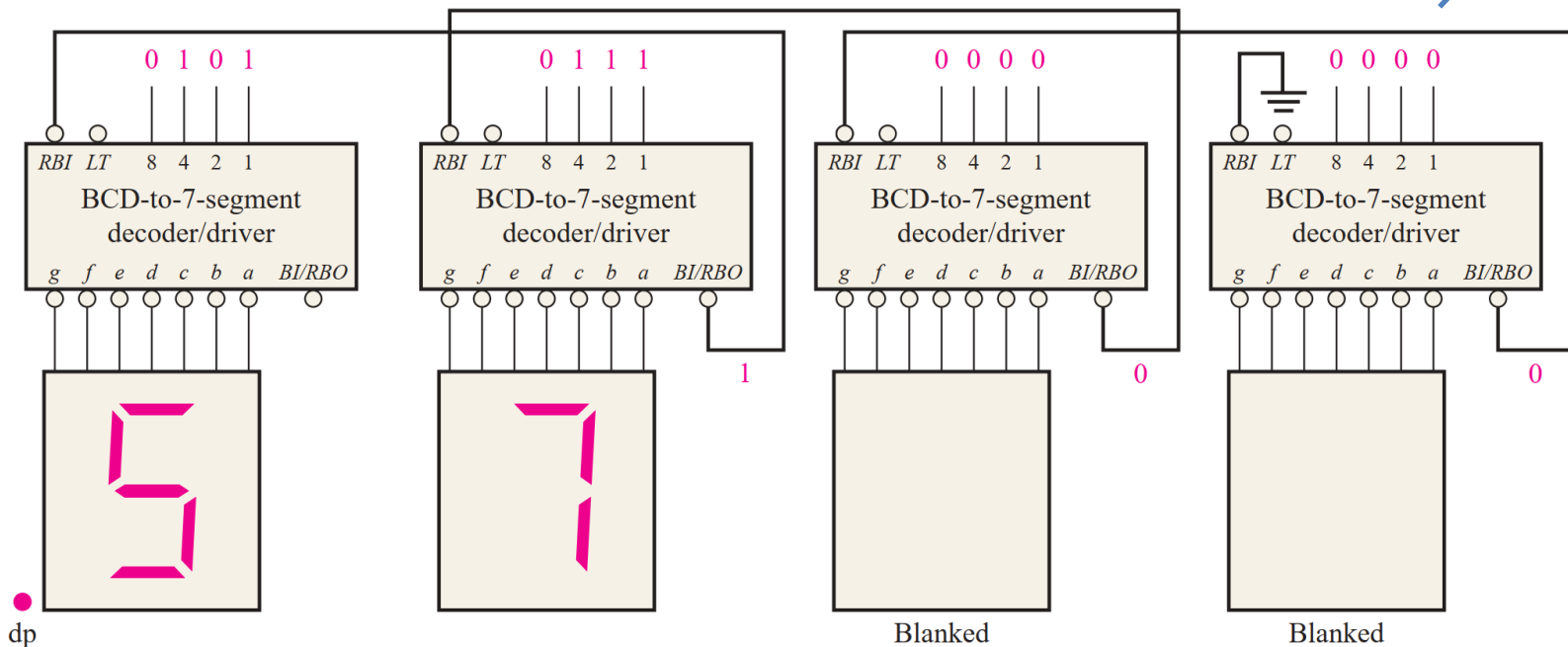
(a) Illustration of leading zero suppression



译码器

• 3. 显示译码器 (BCD-7段译码器)

➤ 附加控制端的功能和用法:



(b) Illustration of trailing zero suppression



译码器

七段显示译码器 7448 的逻辑功能表

功能 (输入)	输入			输入/输出	输出							显示 字形
	<i>LT</i>	<i>RBI</i>	$A_3 A_2 A_1 A_0$	<i>BIRBO</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	
0	1	1	0 0 0 0	1	1	1	1	1	1	1	0	0
1	1	×	0 0 0 1	1	0	1	1	0	0	0	0	1
2	1	×	0 0 1 0	1	1	1	0	1	1	0	1	2
3	1	×	0 0 1 1	1	1	1	1	1	0	0	1	3
4	1	×	0 1 0 0	1	0	1	1	0	0	1	1	4
5	1	×	0 1 0 1	1	1	0	1	1	0	1	1	5
6	1	×	0 1 1 0	1	0	0	1	1	1	1	1	6
7	1	×	0 1 1 1	1	1	1	1	0	0	0	0	7
8	1	×	1 0 0 0	1	1	1	1	1	1	1	1	8
9	1	×	1 0 0 1	1	1	1	1	0	0	1	1	9
10	1	×	1 0 1 0	1	0	0	0	1	1	0	1	A
11	1	×	1 0 1 1	1	0	0	1	1	0	0	1	B
12	1	×	1 1 0 0	1	0	1	0	0	0	1	1	C
13	1	×	1 1 0 1	1	1	0	0	1	0	1	1	E
14	1	×	1 1 1 0	1	0	0	0	1	1	1	1	F
15	1	×	1 1 1 1	1	0	0	0	0	0	0	0	
灭灯	×	×	×	×	×	×	×	×	×	×	×	
灭零	1	0	0 0 0 0	0	0	0	0	0	0	0	0	
试灯	0	×	×	×	×	×	×	×	×	×	×	8

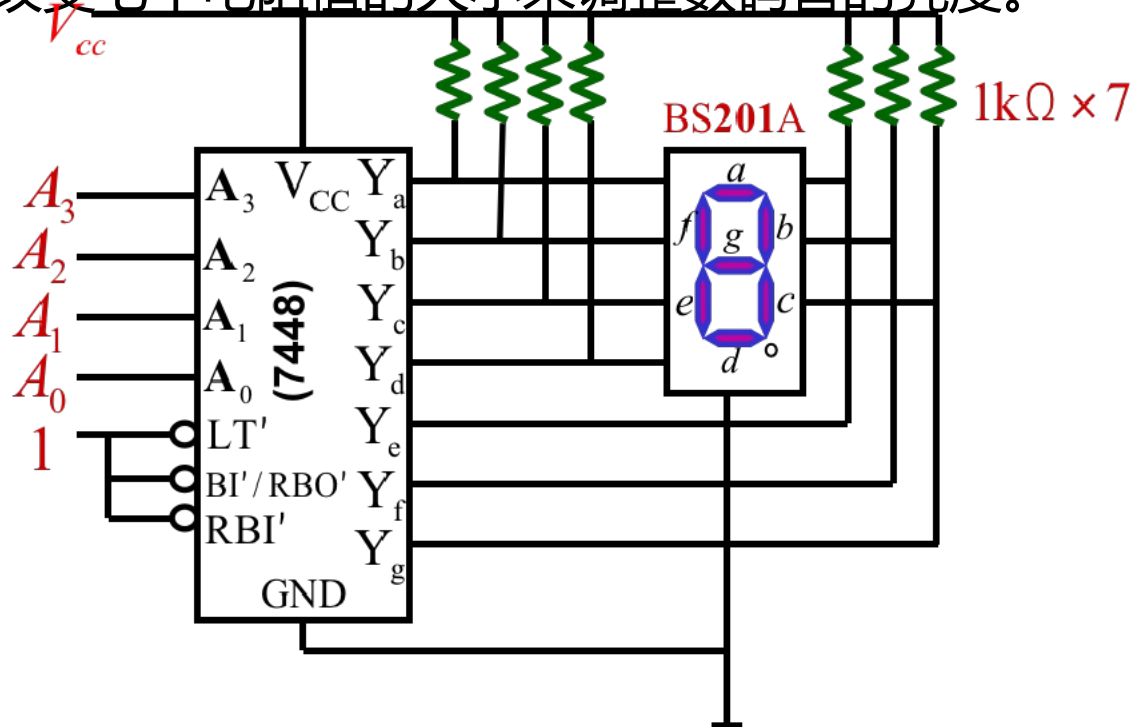


译码器

• 3. 显示译码器 (BCD-7段译码器)

➤ 用74LS48驱动BS201的连接方法：

可通过改变七个电阻值的大小来调整数码管的亮度。



主要内容

1 加法器

2 比较器

3 译码器

4 编码器

5 多路复用/分配器



编码器

- **编码和编码器**

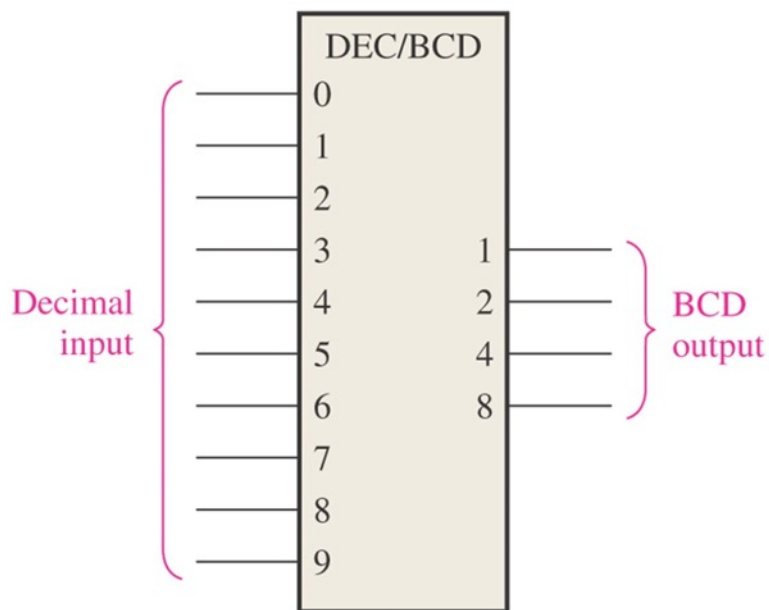
- 把二进制码按一定的规律编排，使每组代码具有一定的含义，称为**编码**。
- 具有编码功能的逻辑电路称为**编码器**。
- 常用的编码器有：**普通编码器和优先编码器**。



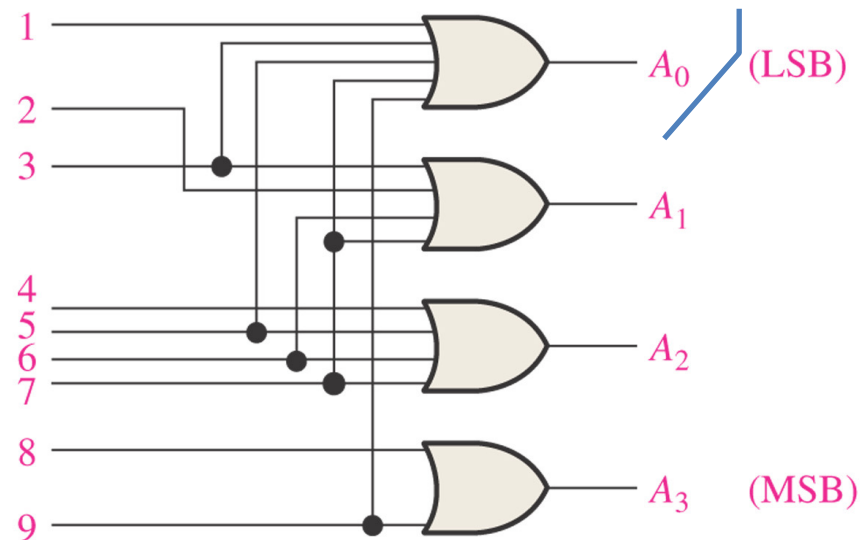
编码器

• 普通编码器

➤ 任何时刻只允许输入一个编码信号，否则输出将发生混乱。



十进制-BCD编码器的逻辑符号



十进制-BCD编码器的逻辑框图

注：不需要0输入



编码器

- **优先编码器**

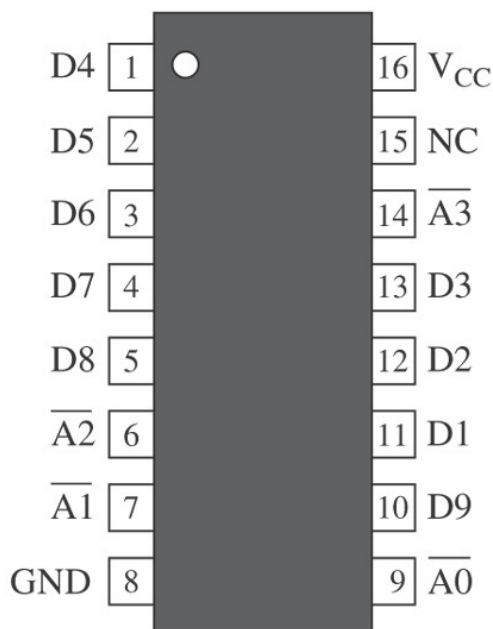
- 在优先编码器电路中，允许同时输入两个以上的编码信号。
- 在设计优先编码器时，已将所有的输入信号按优先顺序排好队。当几个输入信号同时出现时，只对其中优先权最高的一个进行编码。



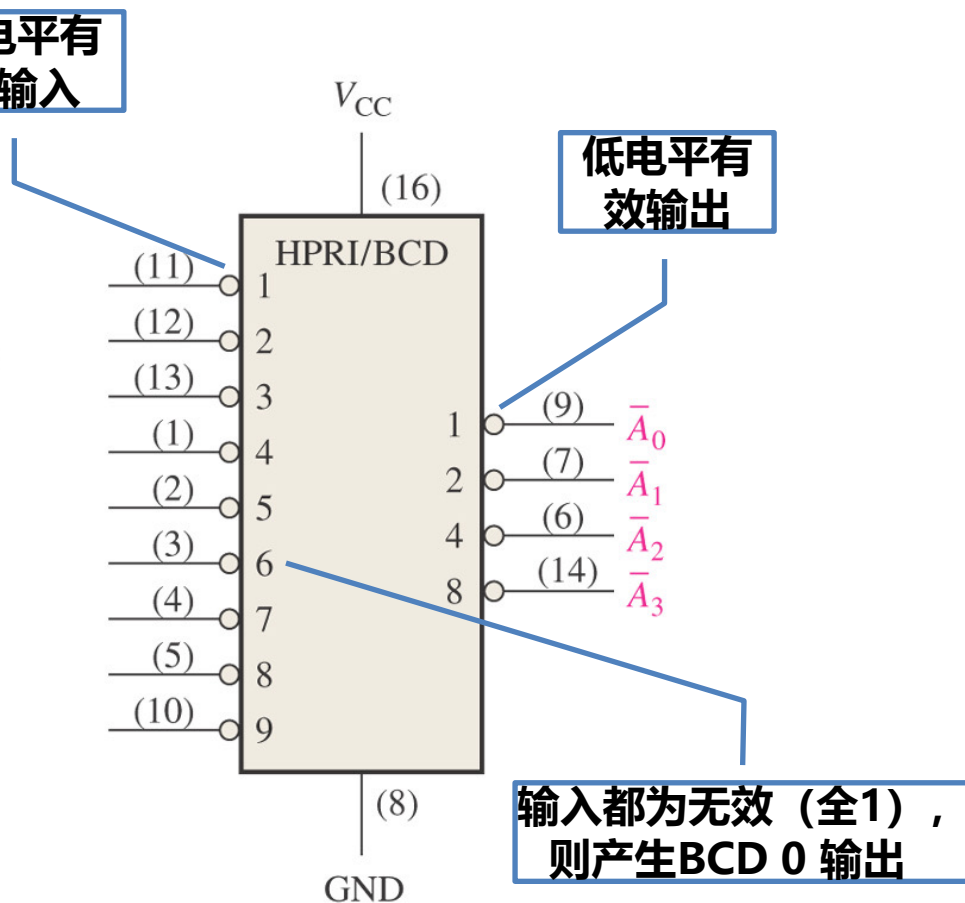
编码器

• 优先编码器

➤ 74HC147 (10-4线)



(a) Pin diagram



(b) Logic diagram

74HC147十进制-BCD优先编码器的引脚图和逻辑符号（HPRI表示最高值输入优先）



编码器

• 优先编码器

➤ 74HC148 (8-3线)

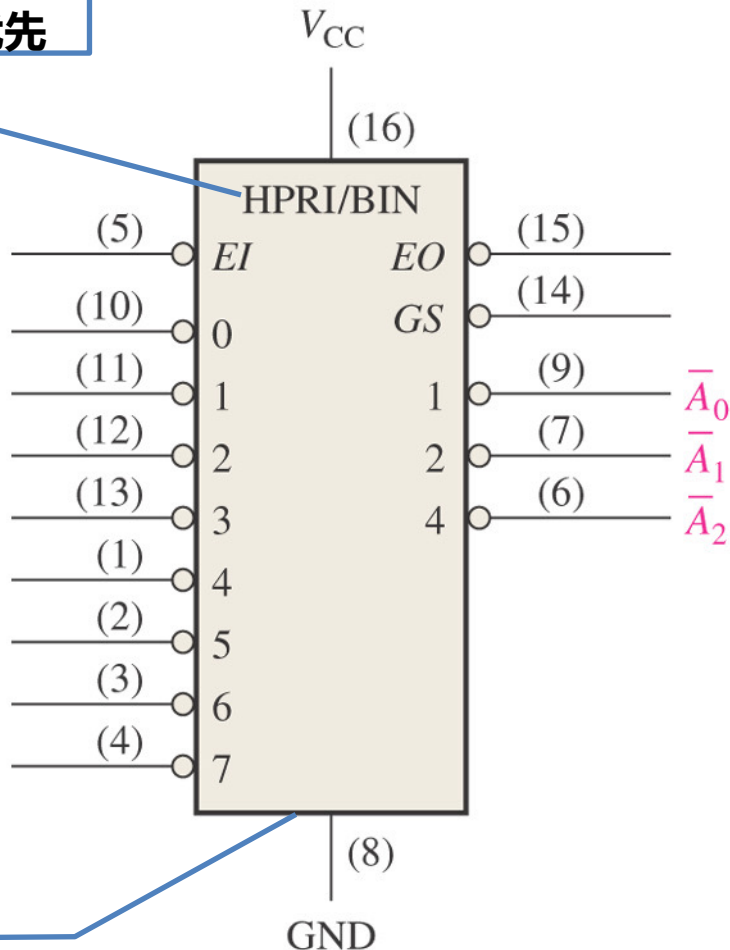
-**EI**: 使能输入, 也叫选通端, 激活芯片, 低电平有效输入

-**EO**: 使能输出, **EI**为低电平且输入都无效时, **EO**=0

-**GS**: **EI**为低电平。且任何一个输入有效时, **GS**=0

功能: 八进制数 (0-7)
转换为3位二进制数

最高值输入
入优先





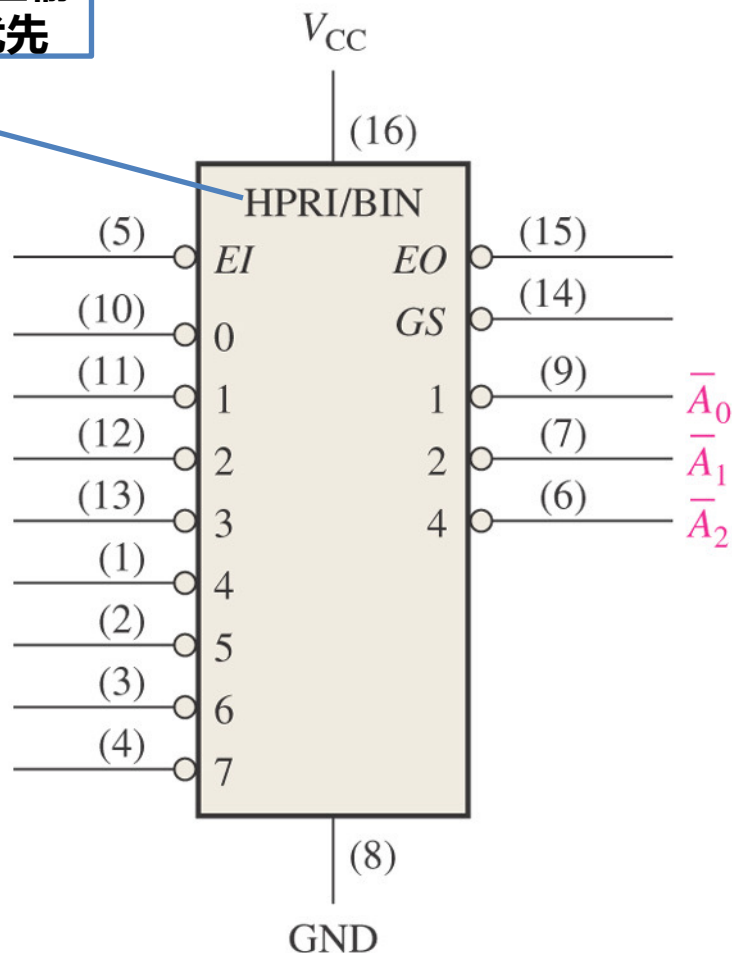
编码器

• 优先编码器

➤ 74HC148 (8-3线)

EO'	GS'	状态
0	0	不可能出现
0	1	工作, 但无输入
1	0	工作, 且有输入
1	1	不工作 ($EI' = 1$)

最高值输入
优先



Inputs									Outputs				
$\overline{EI}$	$\overline{I_0}$	$\overline{I_1}$	$\overline{I_2}$	$\overline{I_3}$	$\overline{I_4}$	$\overline{I_5}$	$\overline{I_6}$	$\overline{I_7}$	$\overline{Y_2}$	$\overline{Y_1}$	$\overline{Y_0}$	$\overline{EO}$	$\overline{GS}$
1	X	X	X	X	X	X	X	X	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	0	1
0	X	X	X	X	X	X	X	0	0	0	0	1	0
0	X	X	X	X	X	X	0	1	0	0	1	1	0
0	X	X	X	X	X	0	1	1	0	1	0	1	0
0	X	X	X	X	0	1	1	1	0	1	1	1	0
0	X	X	X	0	1	1	1	1	1	0	0	1	0
0	X	X	0	1	1	1	1	1	1	0	1	1	0
0	X	0	1	1	1	1	1	1	1	1	0	1	0
0	0	1	1	1	1	1	1	1	1	1	1	1	0

74HC148的真值表



编码器

• 优先编码器

➤ 思考：如何将74HC148级联为16线-4线编码器？

EO'	GS'	状态
0	0	不可能出现
0	1	工作，但无输入
1	0	工作，且有输入
1	1	不工作 ($EI' = 1$)



编码器

注意引脚和优先权的对应

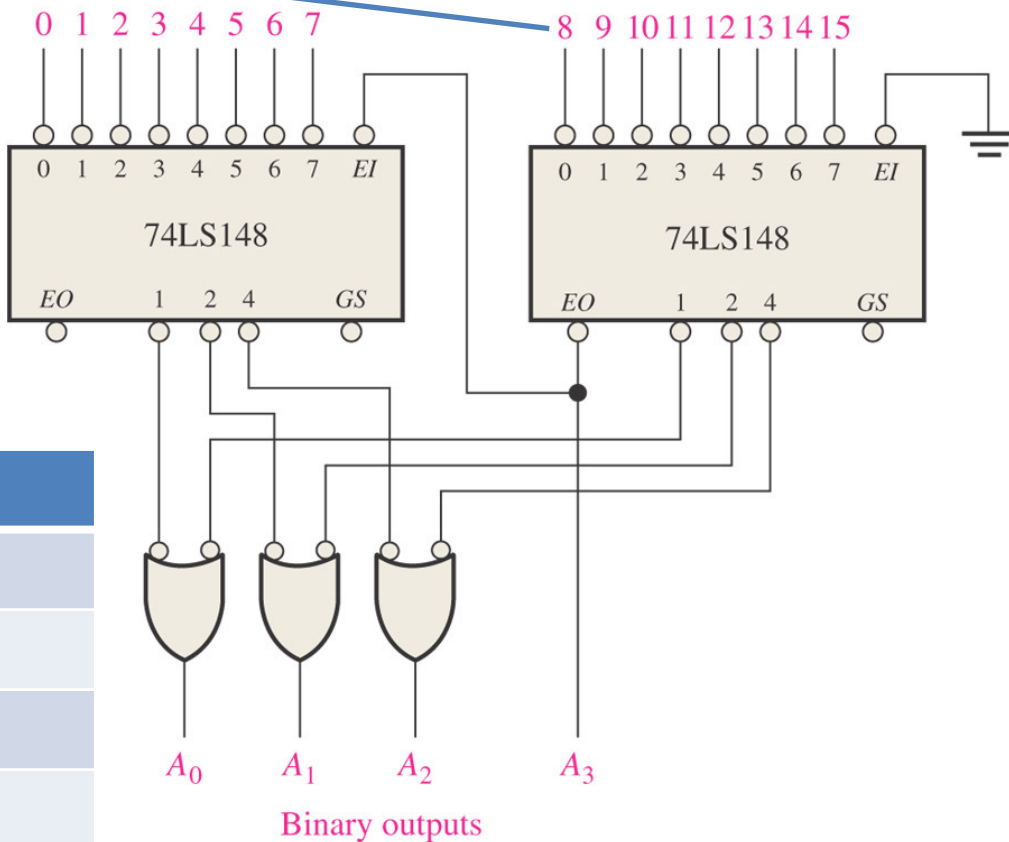
• 优先编码器

➤ 74HC148级联

-高位EO接低位EI

-16线-4线编码器

EO'	GS'	状态
0	0	不可能出现
0	1	工作，但无输入
1	0	工作，且有输入
1	1	不工作 ($EI' = 1$)





代码转换器

• BCD-二进制转换

➤ 使用加法器：BCD位的权所表示的二进制相加

$$\begin{array}{c} \underbrace{1000} \\ 8 \end{array} \quad \begin{array}{c} \underbrace{0111} \\ 7 \end{array}$$

80	40	20	10	8	4	2	1
B_3	B_2	B_1	B_0	A_3	A_2	A_1	A_0



代码转换器

• BCD-二进制转换

➤ 使用加法器：BCD位的权所表示的二进制相加

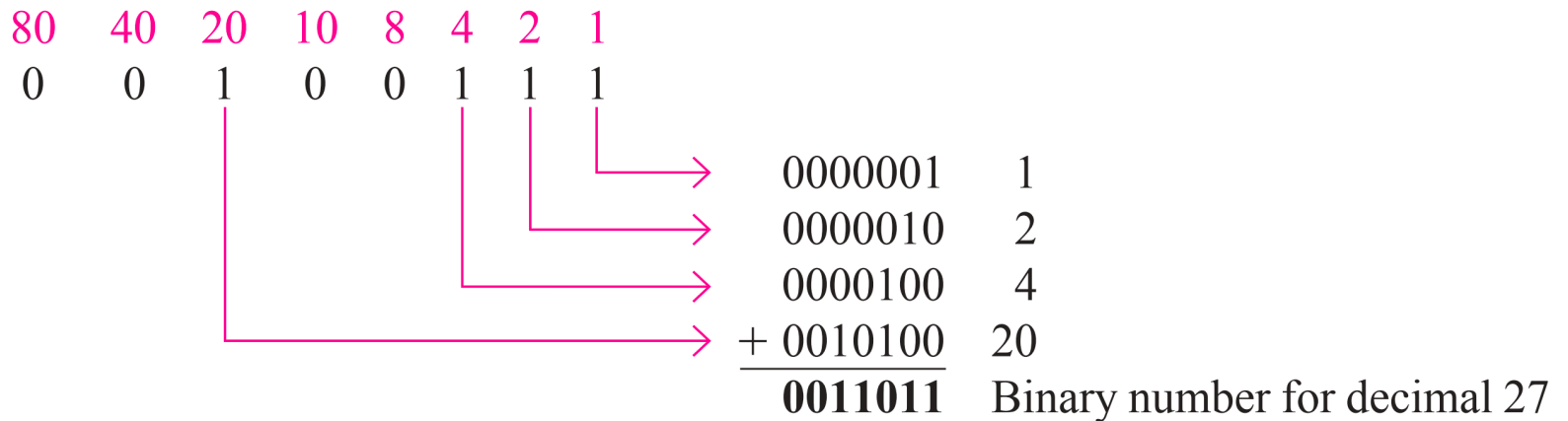
BCD Bit	BCD Weight	(MSB) Binary Representation (LSB)						1
		64	32	16	8	4	2	
A_0	1	0	0	0	0	0	0	1
A_1	2	0	0	0	0	0	1	0
A_2	4	0	0	0	0	1	0	0
A_3	8	0	0	0	1	0	0	0
B_0	10	0	0	0	1	0	1	0
B_1	20	0	0	1	0	1	0	0
B_2	40	0	1	0	1	0	0	0
B_3	80	1	0	1	0	0	0	0



代码转换器

• BCD-二进制转换

➤ **练习：**将BCD码 00100111 (decimal 27)、10011000 (decimal 98) 转换位二进制数

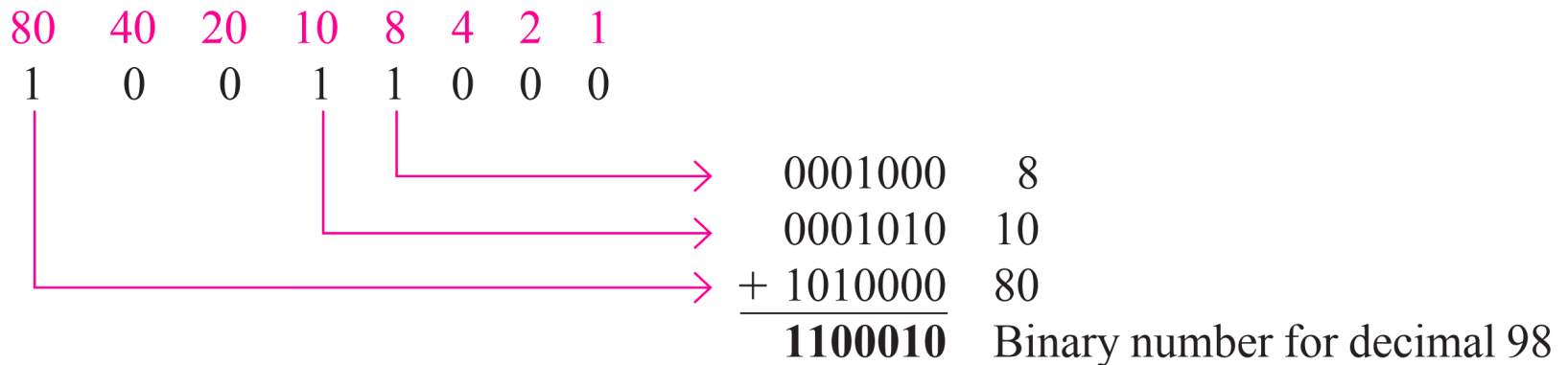




代码转换器

• BCD-二进制转换

➤ **练习：**将BCD码 00100111 (decimal 27)、10011000 (decimal 98) 转换位二进制数

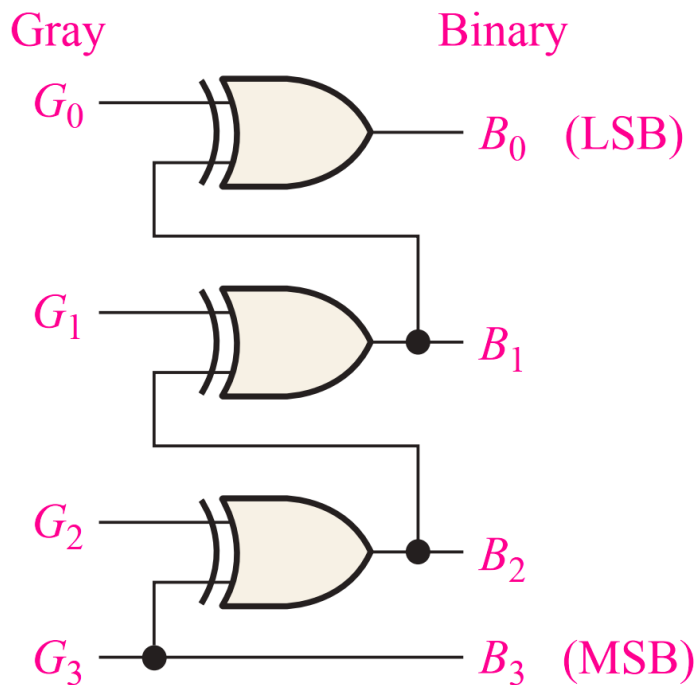
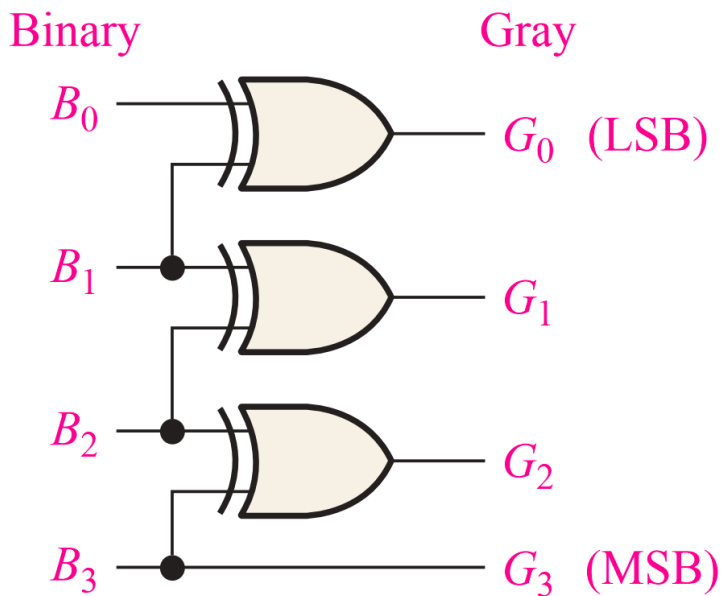
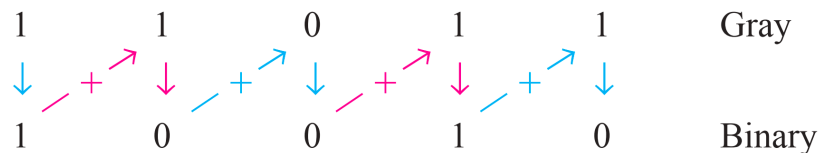
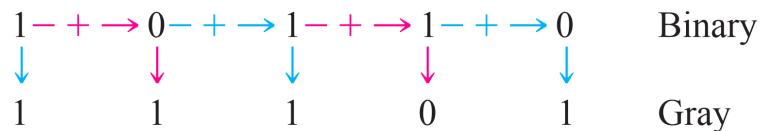




代码转换器

格雷码-二进制和二进制-格雷码转换

➤ 使用异或门



主要内容

1 加法器

2 比较器

3 译码器

4 编码器

5 多路复用/分配器



多路复用器（数据选择器）

• 数据选择器

- 在数字传输过程中，需要从一组数据中选出某一个时，要用到数据选择器，也称为多路开关。
- 数据从几条线路传送到一条线路上

4 选 1 数据选择器功能表

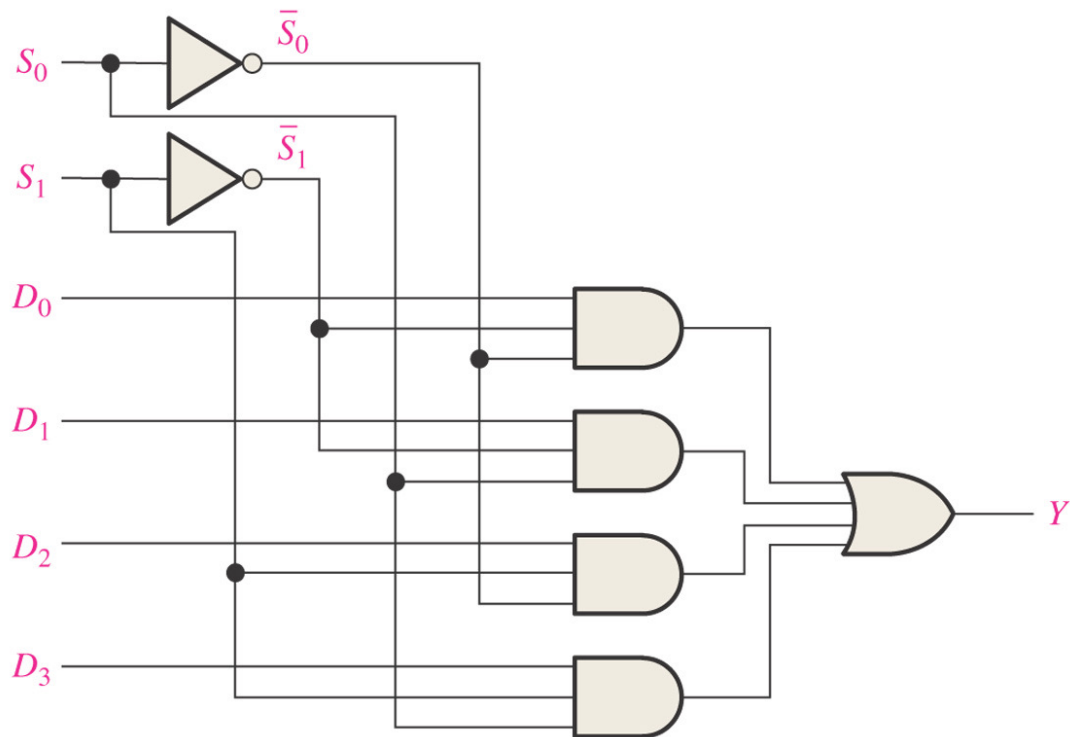
G	输		入				输 出	
	S_1	S_0	D_3	D_2	D_1	D_0	Y	
1	×	×	×	×	×	×	0	
0	0	0	×	×	×	0	0	
			×	×	×	1	1	
	0	1	×	×	0	×	0	
			×	×	1	×	1	
	1	0	×	0	×	×	0	
			×	1	×	×	1	
	1	1	0	×	×	×	0	
			1	×	×	×	1	



多路复用器（数据选择器）

- 数据选择器

$$Y = \overline{S_1}\overline{S_0}D_0 + \overline{S_1}S_0D_1 + S_1\overline{S_0}D_2 + S_1S_0D_3$$

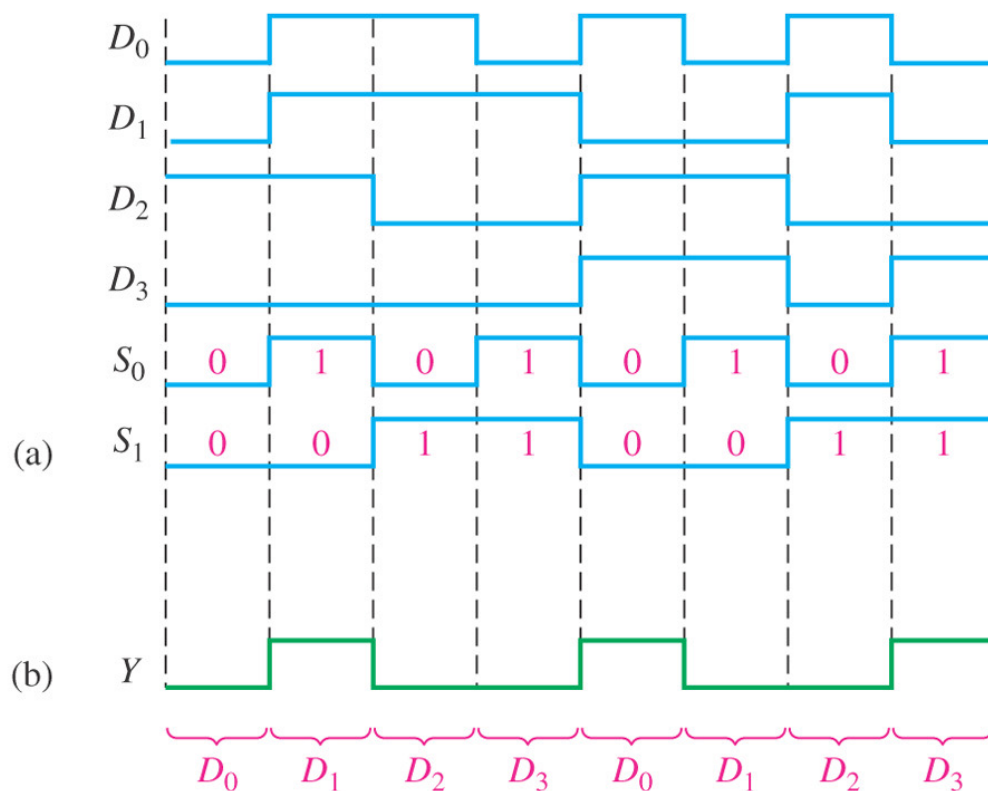




多路复用器（数据选择器）

- 例：从数据选择器的输入波形画输出波形

$$Y = \overline{S_1}\overline{S_0}D_0 + \overline{S_1}S_0D_1 + S_1\overline{S_0}D_2 + S_1S_0D_3$$





多路复用器（数据选择器）

- 数据选择器

➤ 思考：8输入（8条输入线），如何设计数据选择器？

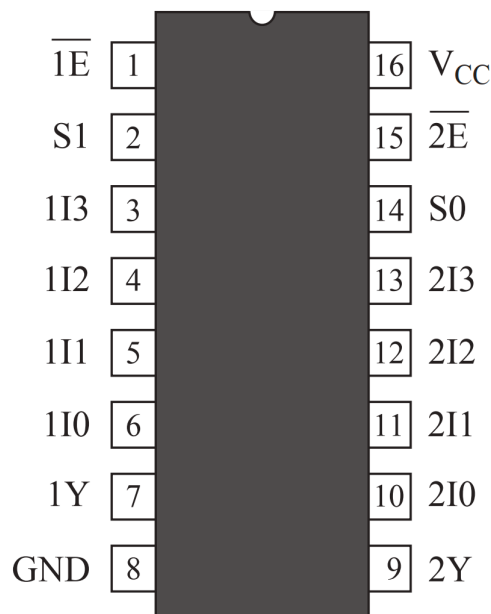
$$Y = (\overline{A_2} \overline{A_1} \overline{A_0})D_0 + (\overline{A_2} \overline{A_1} A_0)D_1 + (\overline{A_2} A_1 \overline{A_0})D_2 + (\overline{A_2} A_1 A_0)D_3 \\ + (A_2 \overline{A_1} \overline{A_0})D_4 + (A_2 \overline{A_1} A_0)D_5 + (A_2 A_1 \overline{A_0})D_6 + (A_2 A_1 A_0)D_7$$



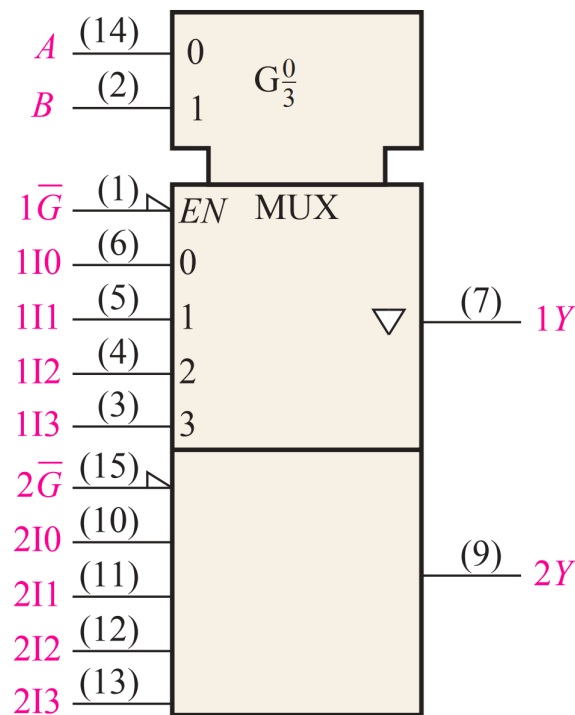
多路复用器（数据选择器）

• 数据选择器

➤ 74HC153 二4输入数据选择器



(a) Pin diagram



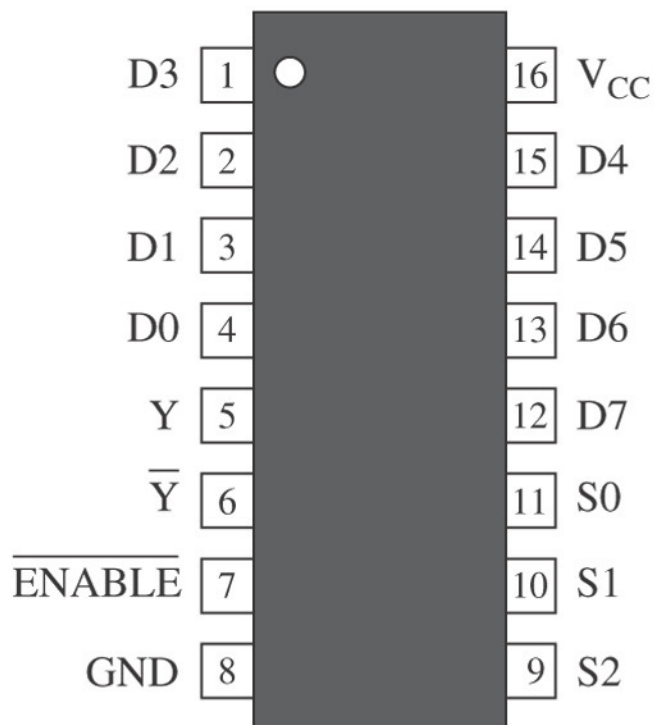
(b) Logic symbol



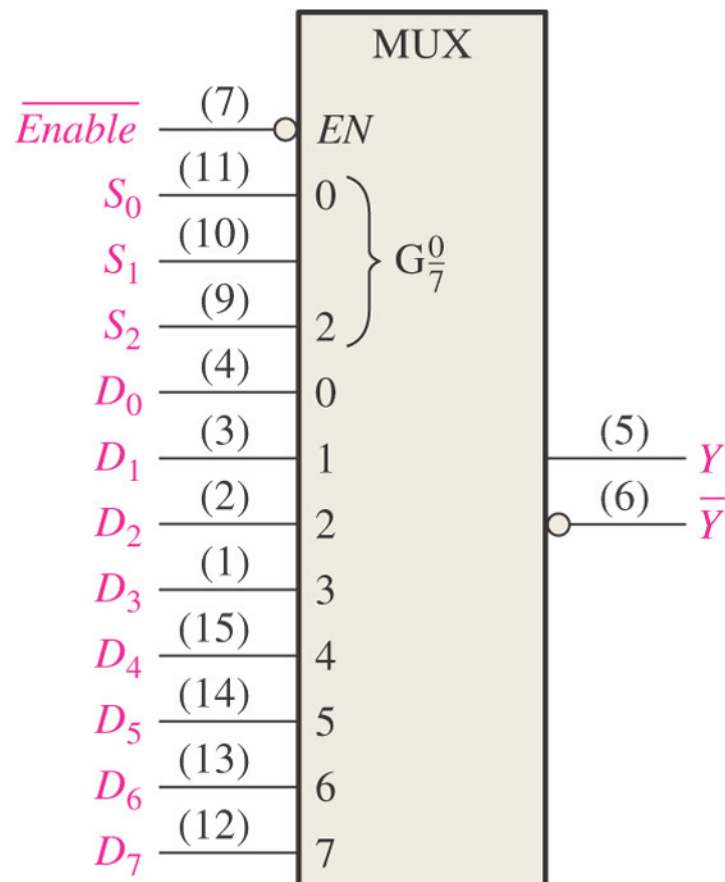
多路复用器（数据选择器）

• 数据选择器

➤ 74LS151八输入数据选择器



(a) Pin diagram



(b) Logic symbol



多路复用器（数据选择器）

- 数据选择器

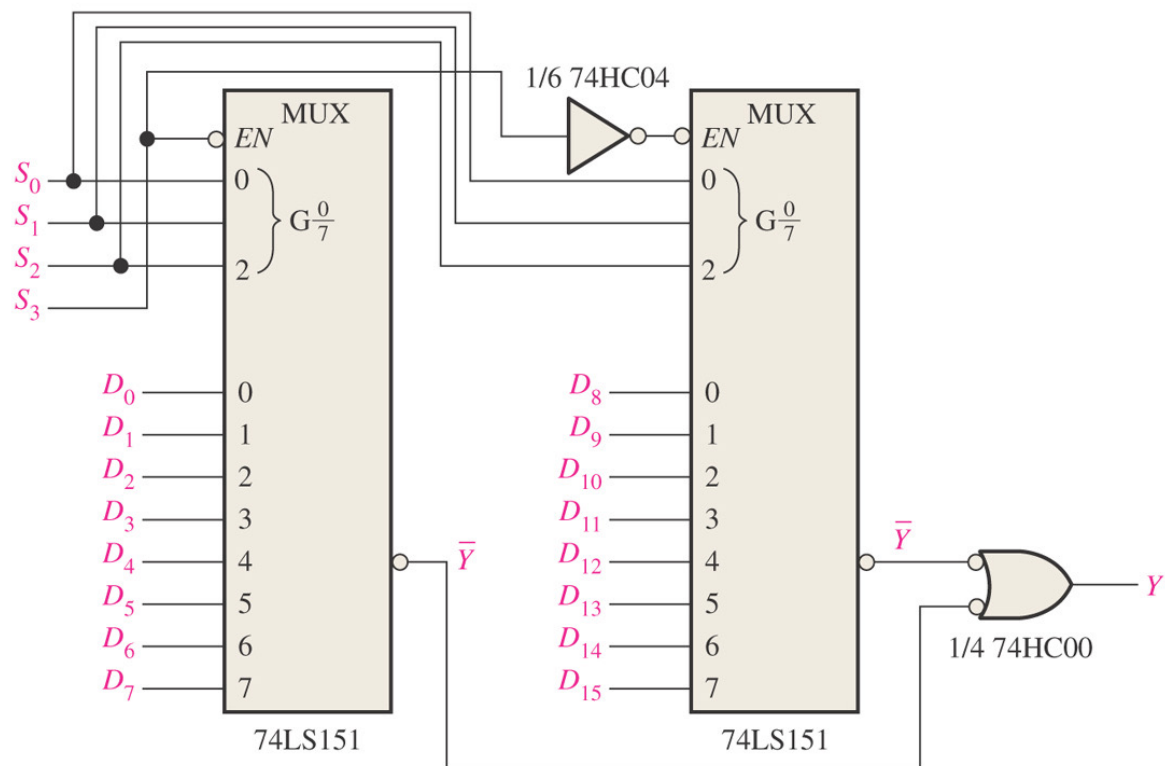
➤ 思考：如何将74LS151级联为16选1数据选择器？



多路复用器（数据选择器）

- 数据选择器

- 74LS151级联：16选1



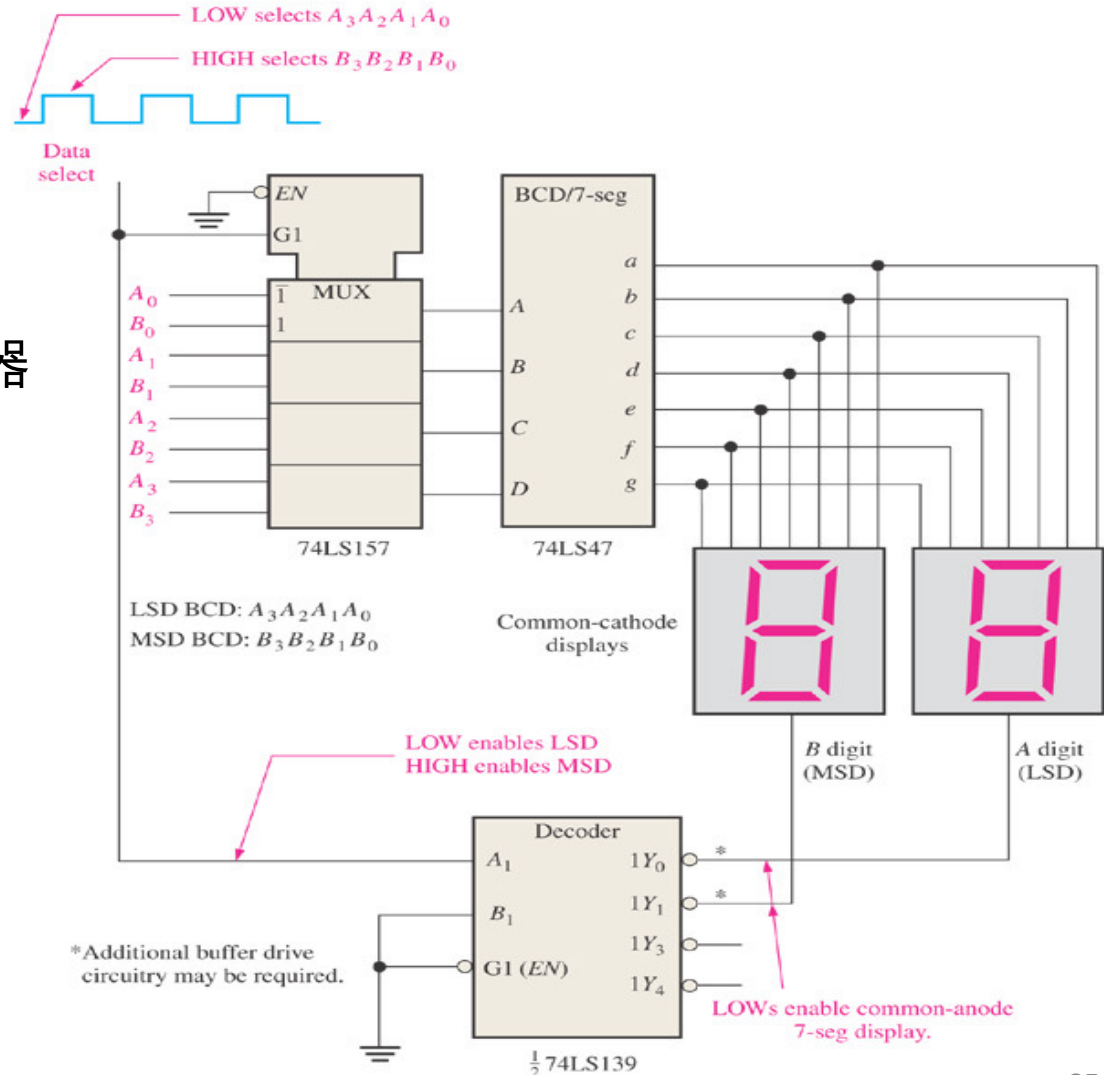


多路复用器（数据选择器）

• 数据选择器

➤ 应用1:

-7段显示多路复用器



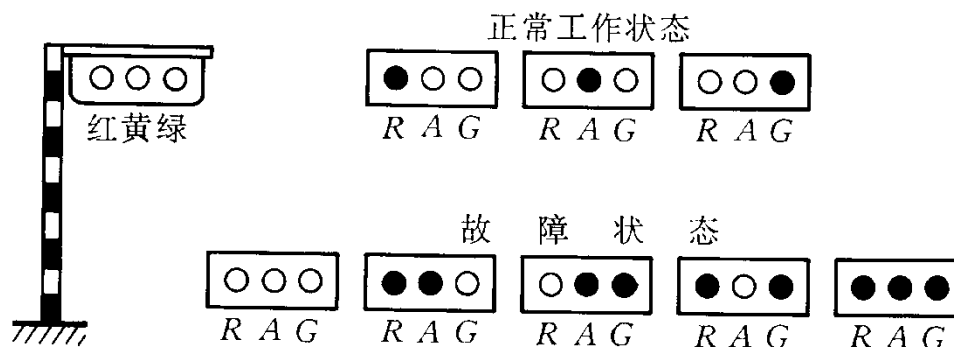


多路复用器（数据选择器）

• 数据选择器

➤ 应用2:

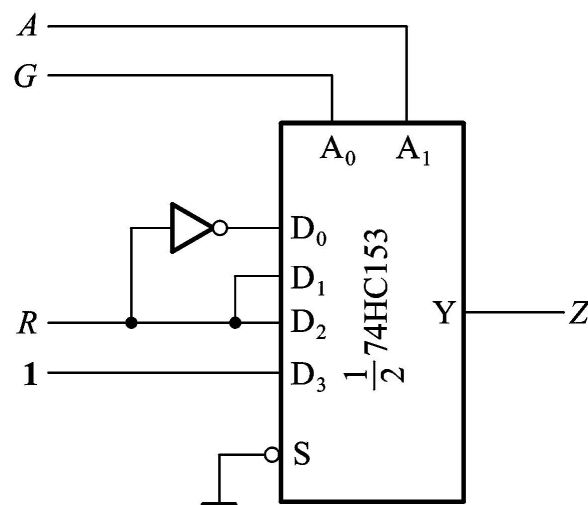
-信号灯故障监控



$$Z = \bar{R}\bar{A}\bar{G} + \bar{R}AG + R\bar{A}\bar{G} + RA\bar{G} + RAG$$

$$= \bar{R}(\bar{A}\bar{G}) + R(\bar{A}G) + R(A\bar{G}) + 1 \cdot (AG)$$

$$Y = D_0(\bar{A}_1\bar{A}_0) + D_1(\bar{A}_1A_0) + D_2(A_1\bar{A}_0) + D_3(A_1A_0)$$





多路复用器（数据选择器）

• 数据选择器

➤ 应用3：函数发生器，用8选1数据选择器产生三变量逻辑

函数 $Z = A'B'C' + AC + A'BC$

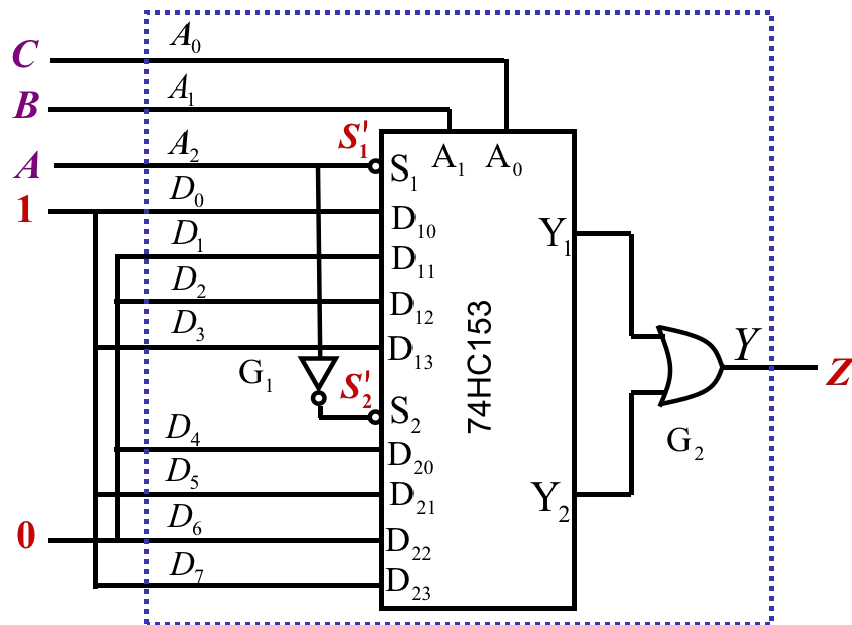
$$A_2 = A$$

$$A_1 = B$$

$$A_0 = C$$

$$D_0 = D_3 = D_5 = D_7 = 1$$

$$D_1 = D_2 = D_4 = D_6 = 0$$

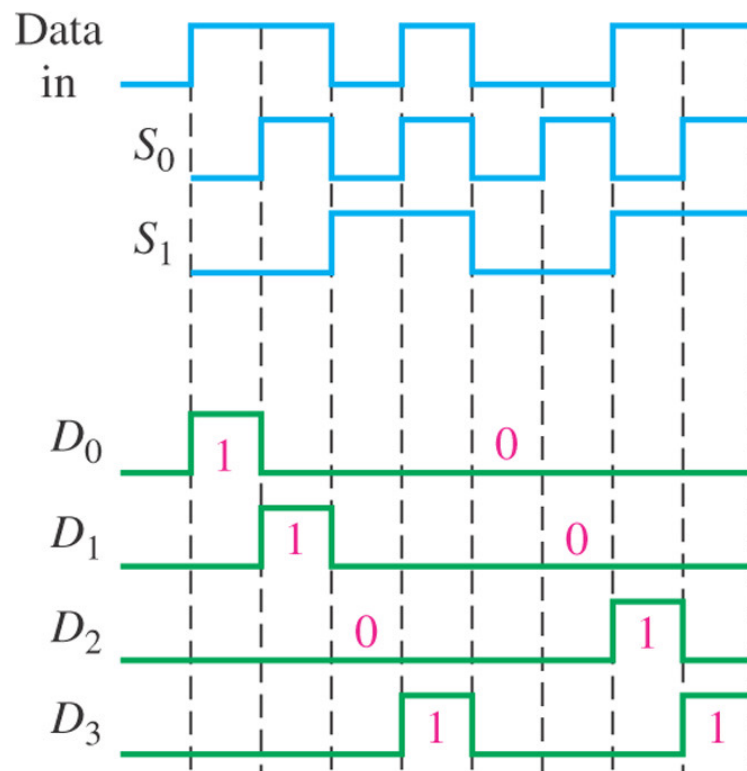
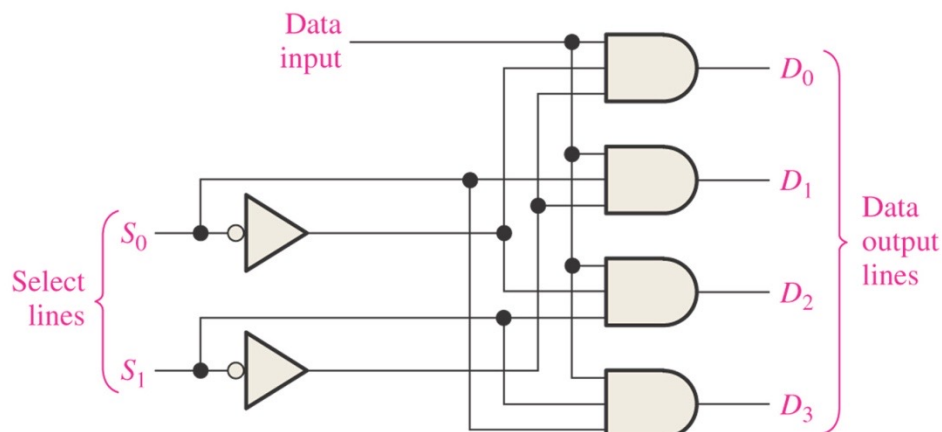




多路分配器

• 多路分配器

➤ 数据从一条线路中被分配到几条线路上

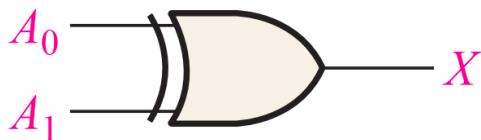




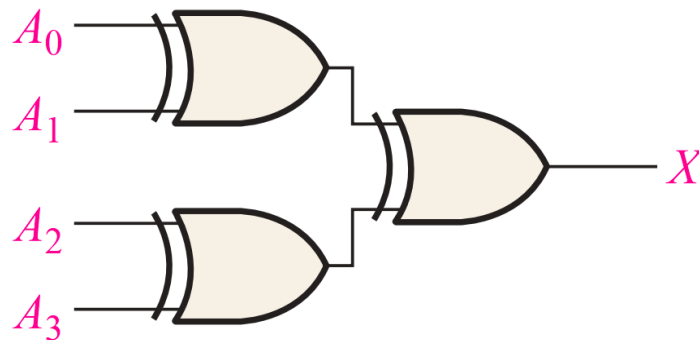
奇偶发生器/校验器

• 基本奇偶逻辑

- 奇偶校验位：指出代码中（含校验位）1的个数是奇/偶
- 偶数个1相加结果为0（进位舍弃），奇数个1相加结果为1



两位的和



四位的和



奇偶发生器/校验器

• 74LS280

➤ 9位奇偶发生器/校验器

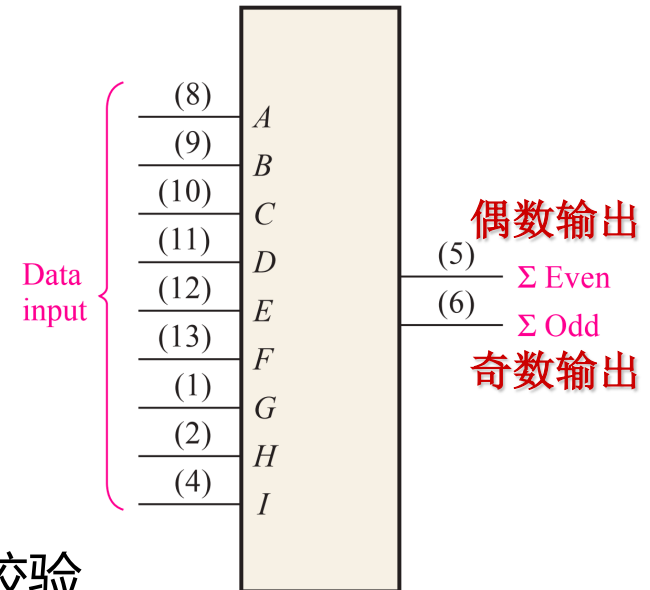
-输入端有偶数个1时，偶数输出高电平，

奇数输出为低电平

-**奇偶校验器**：对一个9位代码进行奇/偶校验

-**偶发生器**：奇偶校验位在**奇数输出**获得

-**奇发生器**：奇偶校验位在**偶数输出**获得



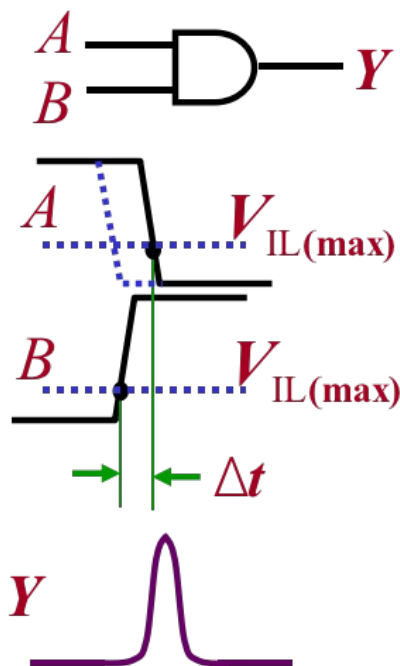
(a) Traditional logic symbol

Number of Inputs A–I that Are High	Outputs	
	Σ Even	Σ Odd
0, 2, 4, 6, 8	H	L
1, 3, 5, 7, 9	L	H

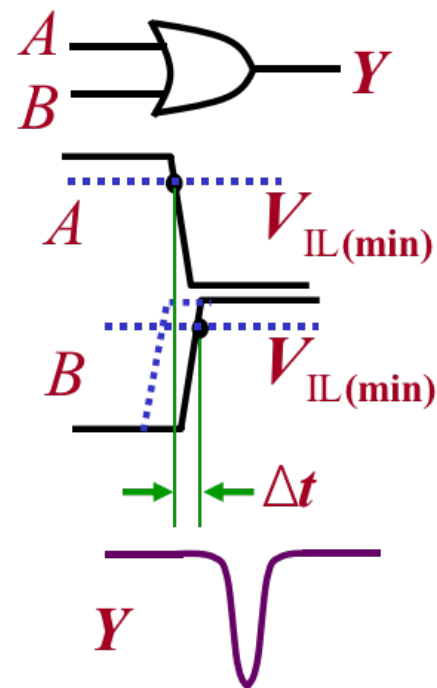
(b) Function table



组合逻辑电路中的竞争-冒险现象



在A从1跳变为0时，B首先上升到 $V_{IL(max)}$ 以上，则在 Δt 内将出现A、B同时高于 $V_{IL(max)}$ 的状态，在输出端产生 $Y=1$ 尖峰脉冲。



在A下降到 $V_{IH(min)}$ 时，B尚未上升到 $V_{IH(min)}$ ，则在 Δt 内将出现A、B同时低于 $V_{IH(min)}$ 的状态，在输出端产生 $Y=0$ 尖峰脉冲。



组合逻辑电路中的竞争-冒险现象

• 竞争

- 把门电路两个输入信号**同时**向相反的逻辑电平跳变，即一个从1变为0，另一个从0变为1的现象叫做竞争。
- 有竞争现象时不一定都会产生尖峰脉冲。只能说只要存在竞争现象，输出就有可能出现违背稳态下逻辑关系的尖峰脉冲。

• 竞争-冒险

- 由于竞争而在电路输出端可能产生尖峰脉冲的现象。



组合逻辑电路中的竞争-冒险现象

• 检查竞争-冒险现象的方法

➤ 在输入变量每次只有一个改变状态的简单情况下，可以通过逻辑函数式，判断组合逻辑电路中是否有竞争-冒险现象。竞争-冒险由于竞争而在电路输出端可能产生尖峰脉冲的现象。

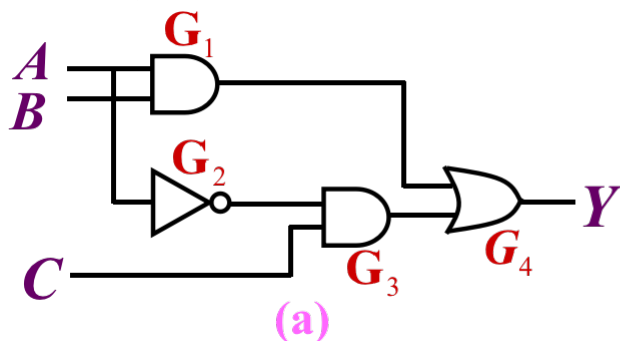
➤ 只要输出端的逻辑函数在一定条件下能简化成：
$$Y = A + A'$$
 或
$$Y = A \cdot A'$$

即可判定存在竞争-冒险。



组合逻辑电路中的竞争-冒险现象

- 例：试判断图示两个电路中是否存在竞争-冒险现象。已知任何瞬间输入变量只可能有一个状态改变。

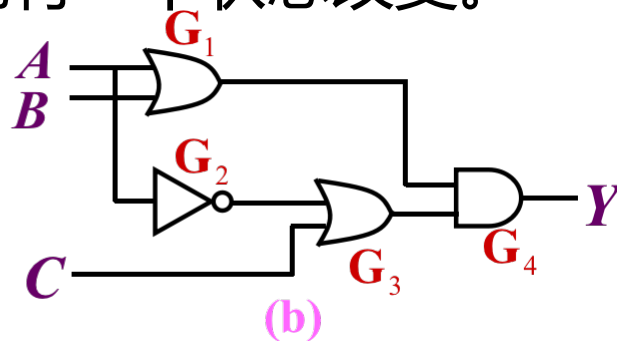


$$Y = AB + A'C$$

当 $B=C=1$ 时

$$Y = A + A'$$

(a) 电路中存在竞争-冒险现象。



$$Y = (A + B) \cdot (B' + C)$$

当 $A=C=1$ 时

$$Y = B \cdot B'$$

(b) 电路中存在竞争-冒险现象。



组合逻辑电路中的竞争-冒险现象

- 这种方法虽然简单，但局限性太大，因为多数情况下输入变量都有两个以上同时改变状态的可能性。

将计算机辅助分析的手段用于分析数字电路以后，为从原理上检查复杂数字电路的竞争-冒险现象提供了有效的手段。通过在计算机上运行数字电路的模拟程序，能够迅速查出电路是否存在竞争-冒险现象。

另一种方法是用实验来检查电路的输出是否有因为竞争-冒险现象而产生的尖峰脉冲。这时加到输入端的信号波形应该包含输入变量的所有可能发生的状态变化。

只有实验检查的结果才是最终的结论。



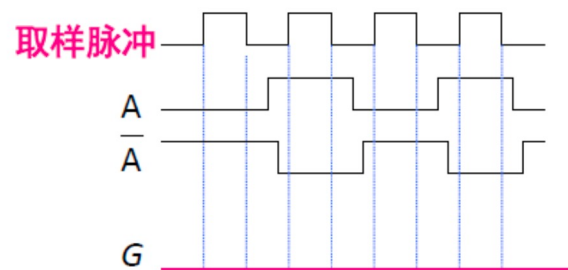
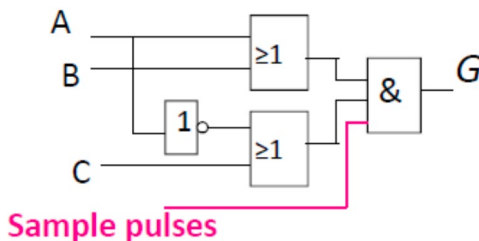
组合逻辑电路中的竞争-冒险现象

• 消除竞争-冒险现象的方法

➤ 1. 接入滤波电容

-在输出端并接一个很小的滤波电容，电容的数值通常在几十至几百皮法的范围内。

➤ 2. 引入选通 (取样) 脉冲



High logic levels of sample pulses appear when stabilized output could be generated, and thus nullifies the glitch in its zero periods 在输出端接取样脉冲，仅在输出处于稳定值时出现。取样脉冲为0期间，输出端信息无效。



组合逻辑电路中的竞争-冒险现象

• 消除竞争-冒险现象的方法

➤ 3. 修改逻辑设计

-在输出逻辑函数式中增加冗余项。

3. Modify logic function

$$\left. \begin{array}{l} F = A + \bar{A} \\ F = A \cdot \bar{A} \end{array} \right\} \text{Harzard}$$

We could introduce redundancy to avoid harzard
引入冗余项，可以消除冒险

$$F = AB + \bar{A}C \quad \longrightarrow \quad F = AB + \bar{A}C + BC$$

↓
No harzard



作业

- 10th, Page 250~253
 - 14
 - 20
 - 28
 - 29
- 11th, Page 214~216
 - 14
 - 20
 - 28
 - 29



课程回顾

- 基本组合逻辑电路
- 组合逻辑的实现
- 从布尔式/真值表到逻辑电路
- 与非门、或非门通用特性
- 具有脉冲波形输入的逻辑电路
- 如何设计组合逻辑?



计算机学院
于帅

